



**INNOPOLIS
UNIVERSITY**

Введение в компьютерное зрение

Илья Афанасьев, к.т.н., ЛИРС

Лекция 4 · Геометрические преобразования и ресемплинг

Dr. Ilya Afanasyev, i.afanasyev@innopolis.ru

«Introduction to Computer Vision» course, Innopolis University (IU), Russia, Fall 2026

Блок I Изображение и его источник	Блок II Операции над изображением	Блок III Признаки	Блок IV Гомография и геометрия	Блок V Видео и движение
L1 Введение. Изображение и видео как данные	L4 Геометрические преобразования	L7 Границы, линии и робастное оценивание	L10 Гомография: оценивание и выравнивание	L13 Видео: движение и оптический поток
L2 Формирование изображения: оптика и сенсор	L5 Пространственная фильтрация	L8 Точки интереса: детекторы	L11 Модель камеры и калибровка	L14 Трекинг и распознавание объектов
L3 Цвет, гистограммы и точечные операции	L6 Частотная область и пирамиды	L9 Дескрипторы и сопоставление	L12 Стерео и эппиплярная геометрия	L15 Синтез: от двух видов к 3D. Итоги

Сегодня — L4: Геометрические преобразования

RECAP

- **Как проще всего выделить объект:** С помощью маски (выделение по цвету через `inRange`), затем убрать мелкий шум («открытие» и «заккрытие») и обвести объект в рамку (`bbox`).
- **Как правильно менять размер (ресайз):** При уменьшении картинки лучше всего работает метод `INTER_AREA`, а для масок (где важны четкие границы) подходит только `INTER_NEAREST`
- **Модель камеры:** Трехмерный мир проецируется на плоский кадр (матрицу пикселей), что подводит к теме гомографии (изменению ракурса/перспективы).

Разбор практических задач (Самопроверка L3)

Поиск красного цвета: Красный цвет в палитре HSV находится на стыке шкалы, поэтому для него всегда нужны **две маски** (самые малые и самые большие значения оттенка *H*)

Выделение мелкого объекта на плавном фоне: Когда стандартный метод Отсу (Otsu) не работает (режет фон пополам), нужен адаптивный порог (`adaptiveThreshold`) или выделение по цвету.

Очистка картинки от шума: Чтобы убрать мелкие точки (2 пикселя) и заделать дыры (6 пикселей), используют морфологические операции (эрозию и дилатацию)

LEARNING OBJECTIVES

- Сдвиг, поворот, аффинное и проективное преобразование одной матрицей 3×3 в **однородных координатах**, а также сколько **степеней свободы** и пар точек нужно каждому
- Сохранения для каждой ступени иерархии: длины, углы, параллельность, прямые линии, двойное отношение
- Почему варп выполняется **обратным отображением**, и как выбрать флаг интерполяции и режим границы под задачу
- `getPerspectiveTransform`, `warpPerspective`, `warpAffine`, `remap`; PSNR, доля совпавших пикселей и ошибка в пикселях
- Когда гомография незаконна: не плоскость, дисторсия, горизонт в кадре, точки на одной прямой

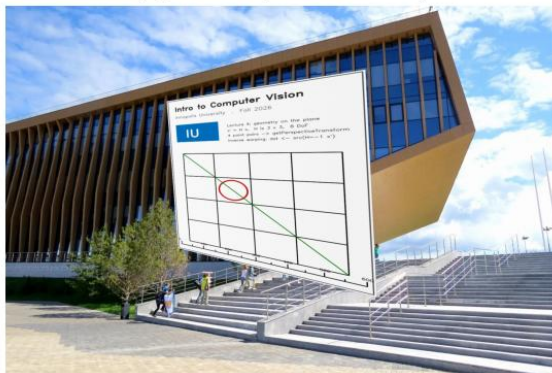
Задача: выпрямить плакат по 4 точкам

МОТИВАЦИЯ

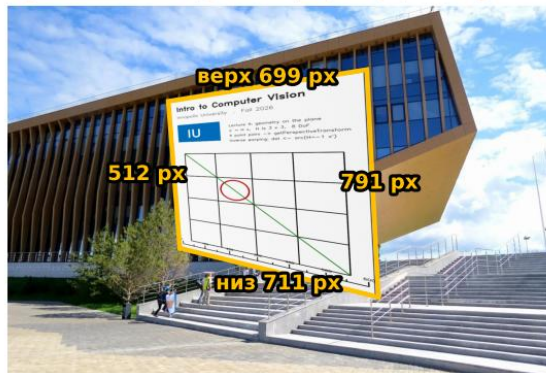
Дано: кадр, где плакат снят под углом. **Найти:** фронтальный вид с правильными пропорциями

- **В лоб не работает:** Из-за эффекта перспективы ближний край кажется больше (791 px), а дальний — меньше (512 px); а на самом деле лист A4 — 0.707. Никакой поворот и `resize` этого не исправят
- **Применения:** · **Робототехника:** bird's-eye view дороги, парковка по разметке, табличка под углом · **МОИИ:** сканер документов и OCR, аугментации, выравнивание данных

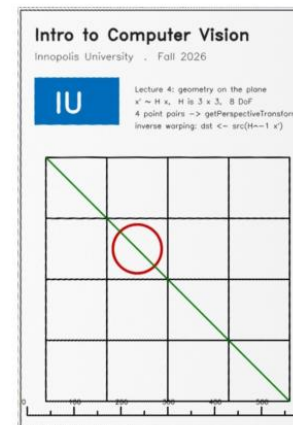
дано: плакат снят под углом
(здание ИУ, 1920 × 1279)



в лоб: стороны 512 и 791 px —
правая на 55 % длиннее, пропорции не узнать



4 точки → гомография →
фронтальный вид 640 × 905:
совпало 99.1 % пикселей



Один кадр: измерить пропорции по нему нельзя; по четырём углам и одной матрице 3×3 плакат возвращается «в лоб»

© фото — здание Университета Иннополис; собственная иллюстрация

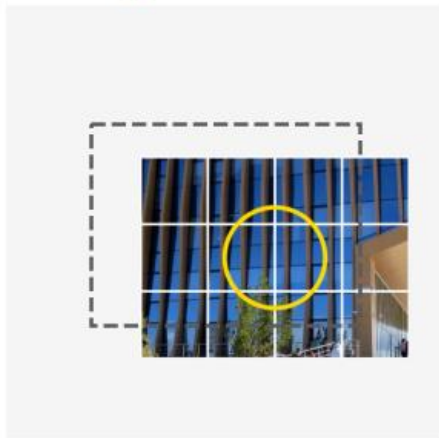
1

Преобразования плоскости

- Однородные координаты; иерархия: сдвиг $\rightarrow$ евклидово $\rightarrow$ подобие $\rightarrow$ аффинное $\rightarrow$ проективное; что каждое сохраняет; композиция; четыре демо

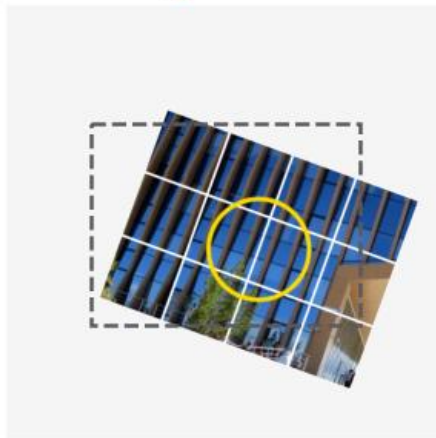
Пять преобразований одной картинкой

сдвиг • 2 DoF



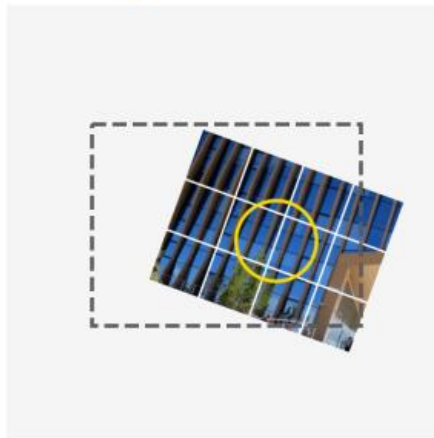
сохраняет всё:
длины, углы, ориентацию

евклидово • 3 DoF



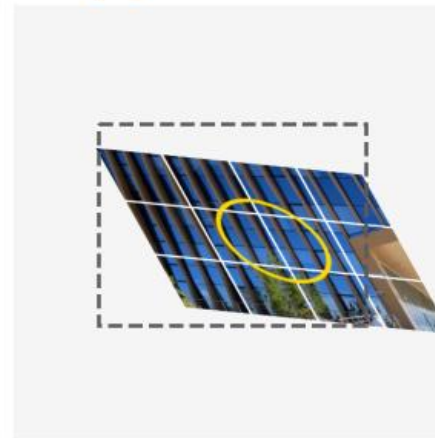
сохраняет длины,
углы, площади

подобие • 4 DoF



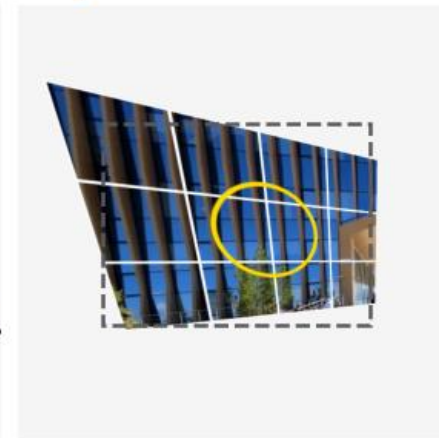
углы, отношения длин;
круг остаётся кругом

аффинное • 6 DoF



параллельность, отношения
площадей; круг → эллипс

проективное • 8 DoF



только прямые и двойное
отношение; параллели сходятся

Один и тот же кусок фасада под пятью степенями иерархии. Слева направо добавляются степени свободы — и теряются свойства: сдвиг сохраняет всё, евклидово — длины и углы, подобие — углы и пропорции, аффинное — параллельность (круг стал эллипсом), проективное — только прямые линии: параллельные сходятся

двойное отношение — это фундаментальный инвариант проективного пространства, который остается абсолютно неизменным. Оно рассчитывается для четырех точек, лежащих на одной прямой line. Если у вас есть четыре последовательные точки (A, B, C, D), то их двойное отношение записывается как:

$$\text{Cross}(A, B, C, D) = \frac{AC}{BC} : \frac{AD}{BD} = \frac{AC \cdot BD}{BC \cdot AD}$$

© фото — здание Университета Иннополис; собственная иллюстрация

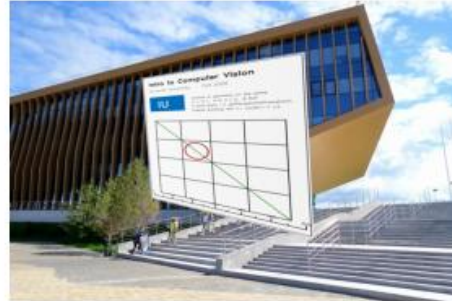
робот: камера
смотрит вперёд...



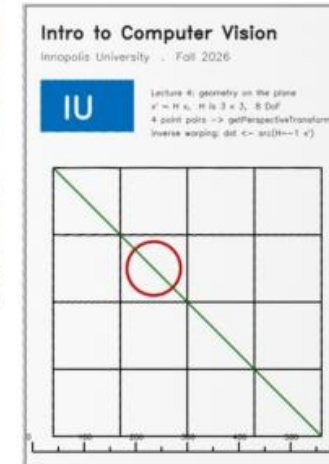
...bird's-eye
view: метры



документ
под углом...



...скан
по 4 точкам



AR: картинка «прилипла»
к плоскости той же H



Три применения одной матрицы: камера робота смотрит вперёд — bird's-eye view переводит кадр в метры; документ под углом — скан по четырём точкам; дополненная реальность — картинка «прилипает» к плоскости той же гомографией, только применённой в обратную сторону

© фото — здание Университета Иннополис; дорога — синтетическая сцена; собственная иллюстрация

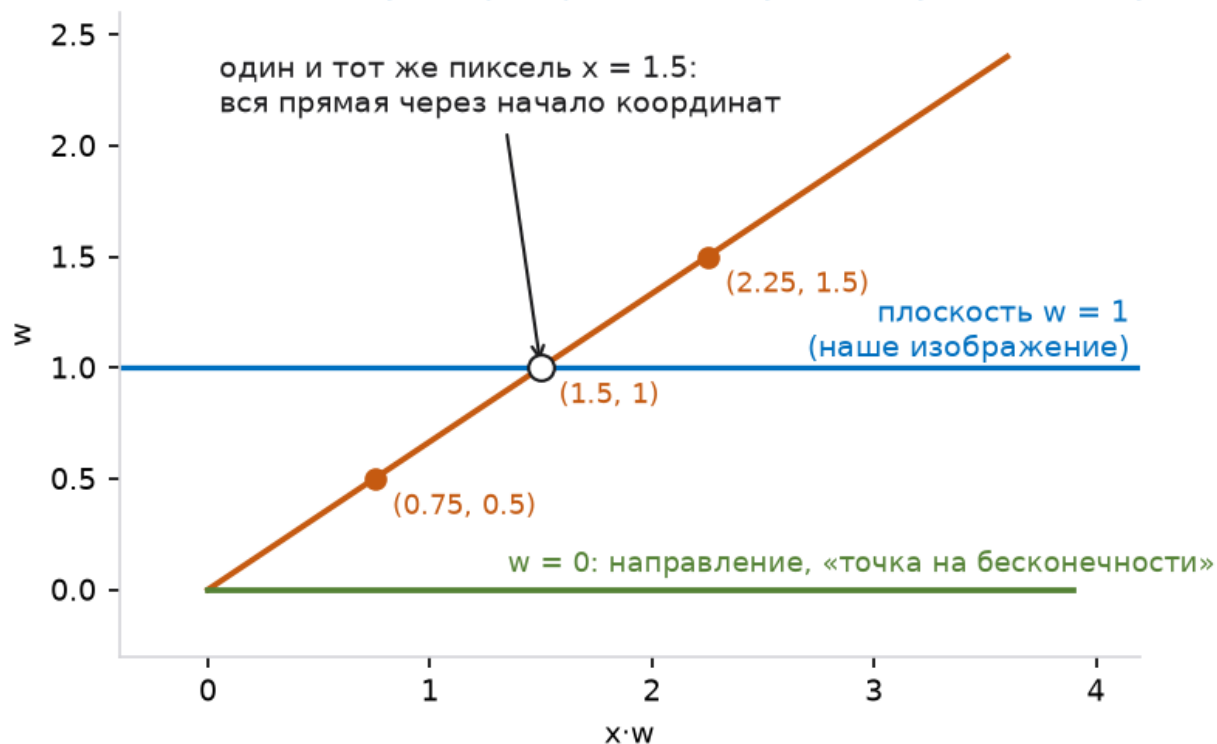
Формально

- **Дано:** изображение $I(x, y)$ и преобразование плоскости $T: (x, y) \rightarrow (x', y')$ — матрицей или парами точек «было $\rightarrow$ стало»
- **Найти:** новое изображение $I'(x', y') = I(T^{-1}(x', y'))$ — те же пиксели на новых местах, значения не меняются (для построения новой картинке используется именно обратное преобразование T^{-1})
- **Как измерить ошибку (Критерии качества):** Для точек: Ошибка измеряется прямо в пикселях (на сколько пикселей алгоритм промахнулся мимо идеальной позиции); Для всей картинки: Полученный результат сравнивают с эталоном через метрику PSNR (индекс пикового отношения сигнала к шуму) и процент пикселей, которые совпали идеально.

Преобразование задаётся числами: у сдвига их два, у гомографии восемь. Сколько чисел — столько пар точек делить на два

Геометрическая операция меняет **места** пикселей, а не их значения; единицы — пиксели, и ошибка тоже измеряется в пикселях

точка плоскости = луч в пространстве (x, y, w) ; $(x, y, w) \sim (\lambda x, \lambda y, \lambda w)$



зачем третья координата

сдвиг на (tx, ty)

без неё: не линейно в (x, y) : $x' = x + tx$

с ней: линейно в $(x, y, 1)$: $\begin{bmatrix} 1 & 0 & tx \\ 0 & 1 & ty \\ 0 & 0 & 1 \end{bmatrix}$

поворот + сдвиг

без неё: две операции: $R \cdot x + t$

с ней: одна матрица 3×3 : $\begin{bmatrix} R & | & t \\ \hline 0 & 0 & 1 \end{bmatrix}$

композиция

без неё: цепочка формул

с ней: произведение матриц: $H = H_2 \cdot H_1$

перспектива

без неё: деление на глубину — нелинейно

с ней: $x' = (h_1^T \tilde{x}) / (h_3^T \tilde{x})$: деление на w в самом конце

Точка плоскости (x, y) записывается тройкой (x, y, w) , причём (x, y, w) и $(\lambda x, \lambda y, \lambda w)$ — одна точка: обратно делим на w .

Зачем: сдвиг становится умножением на матрицу, поворот со сдвигом — одной матрицей $\begin{bmatrix} R & | & t \end{bmatrix}$, композиция — произведением, а перспективное деление откладывается до самого конца. Тройка с $w = 0$ — направление, «точка на бесконечности»: там встречаются параллельные прямые

© собственная иллюстрация

Сдвиг, поворот, масштаб: матрицы 3×3

- **Сдвиг:** $T = \begin{bmatrix} 1 & 0 & t_x; \\ 0 & 1 & t_y; \\ 0 & 0 & 1 \end{bmatrix}$
- **Поворот:** $R = \begin{bmatrix} \cos \theta & -\sin \theta & 0; \\ \sin \theta & \cos \theta & 0; \\ 0 & 0 & 1 \end{bmatrix}$
- **Масштаб:** $S = \text{diag}(s_x, s_y, 1)$
- **Композиция — произведение**, читается **справа налево**: $H = T \cdot R \cdot S$ сначала масштабирует (S), потом поворачивает (R), потом сдвигает (T)
- **Порядок важен:**
 - точка (10, 10) сперва масштаб $\times 2$, потом сдвиг на 100: $T(100, 0) \cdot S(2) \rightarrow (120, 20)$;
 - Сначала сдвиг на 100, потом масштаб $\times 2$: при $S(2) \cdot T(100, 0) \rightarrow (220, 20)$
- **Обращение:** H^{-1} Чтобы вернуть картинку назад, матрицу нужно инвертировать; `cv2.invertAffineTransform` для 2×3
- **Поворот вокруг центра:** По умолчанию формулы крутят картинку вокруг левого верхнего угла (0,0). Чтобы повернуть её вокруг некоего центра, нужно сначала сдвинуть центр в ноль, повернуть, а потом сдвинуть обратно. `getRotationMatrix2D(center, angle, scale)` возвращает 2×3 — верхние две строки $T(c) \cdot R \cdot S \cdot T(-c)$: поворот вокруг центра, а не вокруг (0, 0)

- Basic 2D transformations as 3x3 matrices

$$\begin{bmatrix} x' \\ y' \\ 1 \end{bmatrix} = \begin{bmatrix} 1 & 0 & t_x \\ 0 & 1 & t_y \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x \\ y \\ 1 \end{bmatrix}$$

Translation

перенос

$$\begin{bmatrix} x' \\ y' \\ 1 \end{bmatrix} = \begin{bmatrix} s_x & 0 & 0 \\ 0 & s_y & 0 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x \\ y \\ 1 \end{bmatrix}$$

Scaling

масштабирование

$$\begin{bmatrix} x' \\ y' \\ 1 \end{bmatrix} = \begin{bmatrix} \cos \theta & -\sin \theta & 0 \\ \sin \theta & \cos \theta & 0 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x \\ y \\ 1 \end{bmatrix}$$

Rotation

вращение

$$\begin{bmatrix} x' \\ y' \\ 1 \end{bmatrix} = \begin{bmatrix} 1 & sh_x & 0 \\ sh_y & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x \\ y \\ 1 \end{bmatrix}$$

Shearing

деформация

Степень	Матрица	DoF	Пар точек	Что сохраняет
сдвиг (translation)	$[I \mid \mathbf{t}]$	2	1	всё: длины, углы, ориентацию
евклидово (rigid)	$[R \mid \mathbf{t}]$	3	2	длины, углы, площади
подобие (similarity)	$[sR \mid \mathbf{t}]$	4	2	углы, отношения длин, форму круга
аффинное (affine)	$[A \mid \mathbf{t}]$, A — любая 2×2	6	3	параллельность, отношения площадей, отношения длин на прямой
проективное (homography)	H 3×3 , до масштаба	8	4	прямые линии, двойное отношение, порядок точек на прямой

- Каждая степень включает предыдущую; нужное число пар точек = $\text{DoF} / 2$ — по два уравнения на пару;
- Чем больше степеней свободы (DoF) у матрицы, тем сильнее она может исказить картинку, тем меньше геометрических свойств (инвариантов) сохраняется и тем больше опорных точек нужно алгоритму, чтобы эту матрицу вычислить

сдвиг · 2 DoF

сохраняет всё: длины, углы, ориентацию

евклидово · 3 DoF

сохраняет длины, углы, площади

подобие · 4 DoF

углы, отношения длин; круг остаётся кругом

аффинное · 6 DoF

параллельность, отношения площадей; круг → эллипс

проективное · 8 DoF

только прямые и двойное отношение; параллели сходятся

© фото — здание Университета Иннополис; собственная иллюстрация

2D Affine Transformation

$$\begin{bmatrix} x' \\ y' \\ w \end{bmatrix} = \begin{bmatrix} a & b & c \\ d & e & f \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x \\ y \\ w \end{bmatrix}$$

- Affine transformations:

- Linear transformations
- Translations

- Parallel lines remain parallel

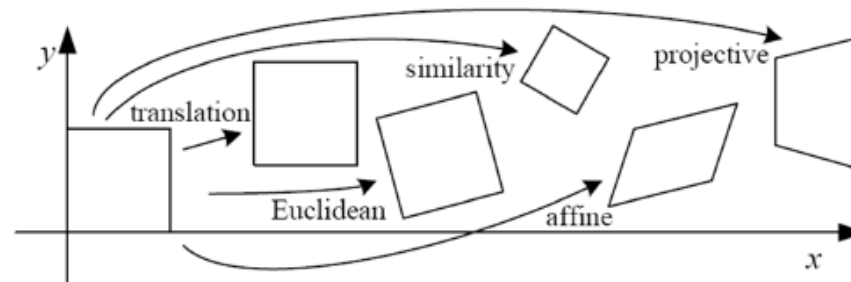
Projective Transformation

$$\begin{bmatrix} x' \\ y' \\ w' \end{bmatrix} = \begin{bmatrix} a & b & c \\ d & e & f \\ g & h & i \end{bmatrix} \begin{bmatrix} x \\ y \\ w \end{bmatrix}$$

- Projective transformations:

- Affine transformations, and
- Projective warps

- Parallel lines do not necessarily remain parallel



1. **Математика:** Правило 3 точек (6 уравнений). В формулах аффинного сдвига есть 6 неизвестных коэффициентов (a, b, c, d, tx, ty). Каждая точка дает две координаты (x) и (y), то есть 2 уравнения. Чтобы однозначно найти всю матрицу в OpenCV с помощью функции `cv2.getAffineTransform`, строго необходимо и достаточно знать координаты всего 3 точек до и после изменения.
2. **Геометрия:** Что происходит с фигурами. Аффинное преобразование — это «всё линейное»: оно может крутить объект, менять масштаб отдельно по вертикали и горизонтали, а также сдвигать слоями (*shear*). Главный инвариант: Параллельные линии всегда остаются параллельными (квадрат превращается в параллелограмм, но не в трапецию; круг становится эллипсом).
3. **Определитель матрицы гомографии $\det A$ и площади:** Число $\det A$ (детерминант линейной части) показывает, во сколько раз изменится площадь объекта. На примере: площадь квадрата уменьшилась на 7% (умножилась на 0.93). Если $\det A < 0$, это значит, что картинку не просто повернули, а отзеркалили.
4. **Ограничение метода:** аффинного преобразования недостаточно для выпрямления плакатов, снятых под углом, потому что оно принципиально не умеет сводить параллельные линии в точку схода (для этого нужна гомография с её 8 параметрами и 4 точками).

Аффинное — «всё линейное»: скос и разномасштабность есть, но параллельные линии не сходятся. Для плаката под углом его мало

- $x' = (h_{11}x + h_{12}y + h_{13}) / (h_{31}x + h_{32}y + h_{33})$, $y' = (h_{21}x + h_{22}y + h_{23}) / (h_{31}x + h_{32}y + h_{33})$ — в однородных координатах это просто $\tilde{x}' \sim H \cdot \tilde{x}$
- 9 элементов, общий множитель не важен $\rightarrow$ **8 DoF**, четыре пары точек:
`cv2.getPerspectiveTransform(src4, dst4)` — точное решение
- **Что физически:** h_{31} , h_{32} — «перспективные» члены: знаменатель зависит от положения, дальняя часть сжимается; при $h_{31} = h_{32} = 0$ это аффинное
- **Когда работает:** сцена — **плоскость** (плакат, дорога, стена) или камера только **поворачивается** без смещения (панорама, L10)

Гомография (homography) — самое общее преобразование, сохраняющее прямые линии; в OpenCV — `warpPerspective`

Параллельные прямые сходятся

Параллельные на плакате линии сетки в кадре сходятся: гомография не сохраняет параллельность; вертикали почти параллельны — их точка схода в (2522, 25351)

Точка схода (Vanishing Point):

- В реальности сетка на плакате состоит из идеально параллельных линий. Но на фотографии FullHD эти линии наклонены друг к другу.
- Если продлить линии плаката влево за пределы экрана, они пересекутся в координате **(-672, 526)**. Это и есть точка схода.



Горизонтальные линии сетки плаката параллельны на самом плакате, а в кадре продолжаютс в одну точку — точку схода **(-672, 526)** за левым краем кадра. Вертикали почти параллельны: их точка схода в (2522, 25352), практически на бесконечности. Точка схода — образ точки $w = 0$

© фото — здание Университета Иннополис; собственная иллюстрация

Пример на числах: H нашего плаката

- Углы плаката в кадре: $(560, 330)$, $(1250, 220)$, $(1290, 1010)$, $(600, 840) \rightarrow$ углы листа $(0, 0)$, $(639, 0)$, $(639, 904)$, $(0, 904)$
- $H = \text{getPerspectiveTransform}(\text{src}, \text{dst})$ — матрица «кадр $\rightarrow$ лист», нормирована так, что $h_{33} = 1$; H^{-1} — «лист $\rightarrow$ кадр»
- **Проверка:** $H \cdot (560, 330, 1)$ даёт $(0, 0)$ с точностью 10^{-6} — четыре опорные точки система воспроизводит точно, ошибка возникает только в **других** точках
- **Перспективные члены** $h_{31} = 0.0014$, $h_{32} = -0.0002$: знаменатель w у левого края плаката ≈ 1.7 , у правого ≈ 2.65 — отношение **1.55**, ровно $791 / 512$: вот откуда разница сторон

Числа пересчитывает демо 1.2; на слайде — порядок величин

Композиция: вокруг чего поворачиваем

`getRotationMatrix2D(центр, 30°)` + тот же `dsize`:
углы отрезаны — потеряно 18 % пикселей



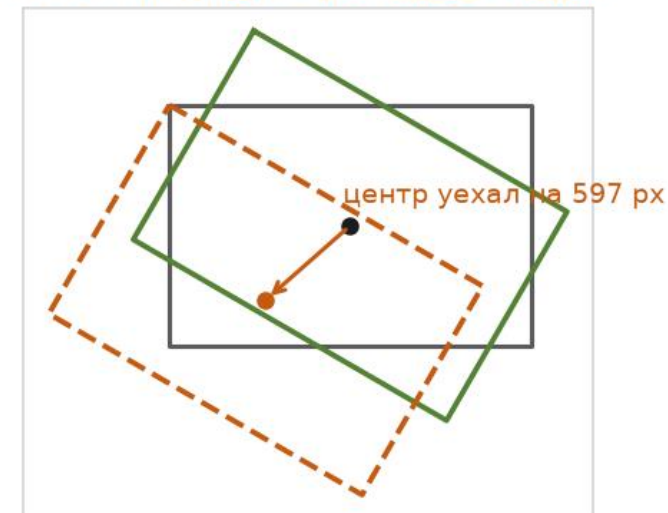
Если вы просто поворачиваете картинку и оставляете старый размер кадра, углы изображения выйдут за границы экрана и безвозвратно удалятся

`dsize` по поворнутым углам: 2302×2068 ,
сдвиг `M[:, 2]` на разницу центров — потеряно 0 %



Чтобы не потерять данные, нужно заранее рассчитать **увеличенный размер холста** по координатам поворнутых углов и сдвинуть матрицу на разницу центров

$R \cdot T \neq T \cdot R$: вокруг чего поворачиваем



- исходный кадр
- поворот вокруг центра
- - поворот вокруг (0, 0)

`getRotationMatrix2D(центр, 30°)` с прежним `dsize` отрезает углы: потеряно 18 % пикселей. Правильно: `dsize` по поворнутым углам — 2302×2068 — и сдвиг `M[:, 2]` на разницу центров. Справа: поворот вокруг (0, 0) вместо центра уносит центр кадра на 597 px — $R \cdot T \neq T \cdot R$

© фото — здание Университета Иннополис; собственная иллюстрация

- **Время работы (Производительность)** на кадре 1920×1279 :
 - Базовые операции деформации (warpAffine, warpPerspective, remap) на стандартном кадре (Full HD) с линейной интерполяцией работают сверхбыстро — всего 3–4 миллисекунды. Это с запасом укладывается в реальное время (быстрее стандартных 30 FPS, где на кадр дается 33 мс).
 - Однако выбор «тяжелой» интерполяции сильно замедляет процесс: кубическая (INTER_CUBIC) требует уже 8 мс, а Ланцош (INTER_LANCZOS4) — целых 19 мс, что может стать критичным для продакшена.
- **Точность и чувствительность к шуму:**
 - Малейшая погрешность или человеческий фактор при разметке углов (шум всего в ± 2 пикселя при клике мышью) приводит к тому, что выпрямленный плакат «поплывет» — узлы его сетки сместятся уже на **4–7 пикселей**.
 - Если тот же самый объект на фото изначально мелкий (вдвое меньше в кадре), эта ошибка драматически возрастает до **12 пикселей**. **Вывод:** чем дальше/меньше объект, тем чувствительнее гомография к ошибкам разметки.

- **Избыточность размеров холста:**
 - Поворот картинки без потерь (как на прошлом слайде) заставляет раздувать холст до 2302x2068. Это удваивает объем обрабатываемых пикселей, причем **48% из них будут абсолютно пустыми** (черными углами). Это плата за сохранение информации.
- **Отрезвление метриками качества (PSNR):**
 - Метрика сходства изображений PSNR очень чувствительна к геометрии. Если картинку просто сдвинуть всего на **1 пиксель** в сторону, метрика PSNR упадет до **26 дБ** — математически это выглядит как сильнейшее ухудшение качества и сильные артефакты сжатия, хотя визуально картинка почти не изменилась.

Что такое гомография?

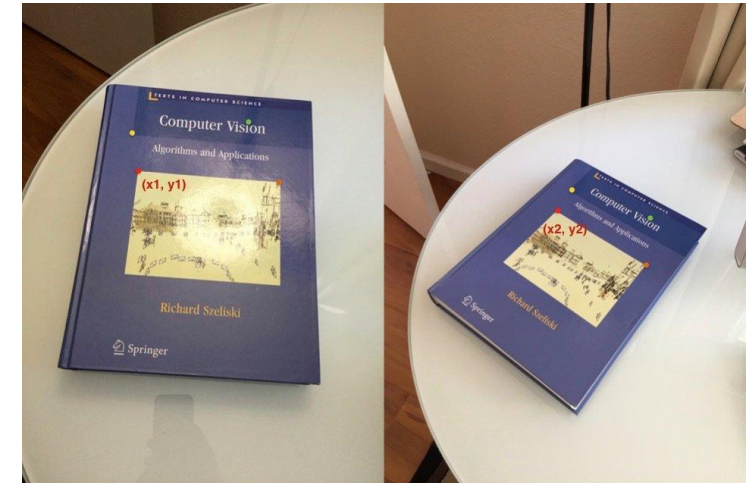
- Рассмотрим два изображения плоскости (верхняя часть книги). Красные, зеленые, желтые и оранжевые точки представляют одну и ту же физическую точку на двух изображениях (т.е. соответствующие точки).
- Гомография — это преобразование (матрица 3×3), которое отображает точки на одном изображении в соответствующие точки на другом изображении.

$$H = \begin{bmatrix} h_{00} & h_{01} & h_{02} \\ h_{10} & h_{11} & h_{12} \\ h_{20} & h_{21} & h_{22} \end{bmatrix}$$

- Рассмотрим набор соответствующих красных точек: (x_1, y_1) на первом изображении и (x_2, y_2) на втором изображении. Тогда гомография H отображает их следующим образом:

$$\begin{bmatrix} x_1 \\ y_1 \\ 1 \end{bmatrix} = H \begin{bmatrix} x_2 \\ y_2 \\ 1 \end{bmatrix} = \begin{bmatrix} h_{00} & h_{01} & h_{02} \\ h_{10} & h_{11} & h_{12} \\ h_{20} & h_{21} & h_{22} \end{bmatrix} \begin{bmatrix} x_2 \\ y_2 \\ 1 \end{bmatrix}$$

- Вышеуказанное уравнение верно для ВСЕХ наборов соответствующих точек, если они лежат на одной плоскости в реальном мире. Другими словами, вы можете применить гомографию к левому изображению, и книга на левом изображении будет выровнена с книгой на правом изображении.



If apply homography to
the left image

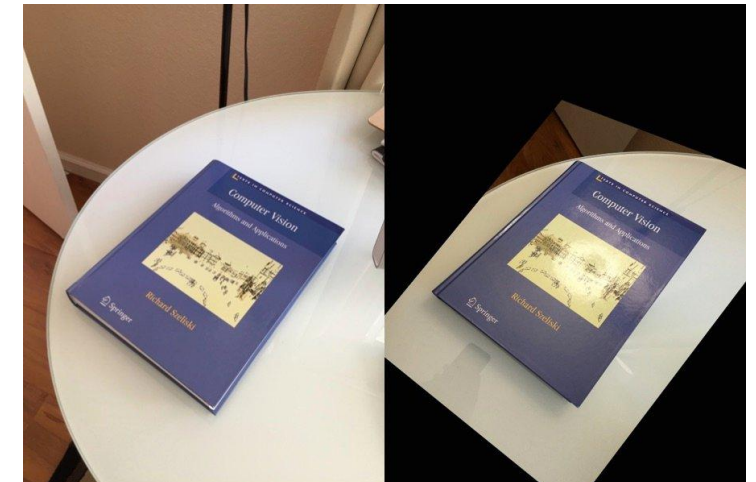
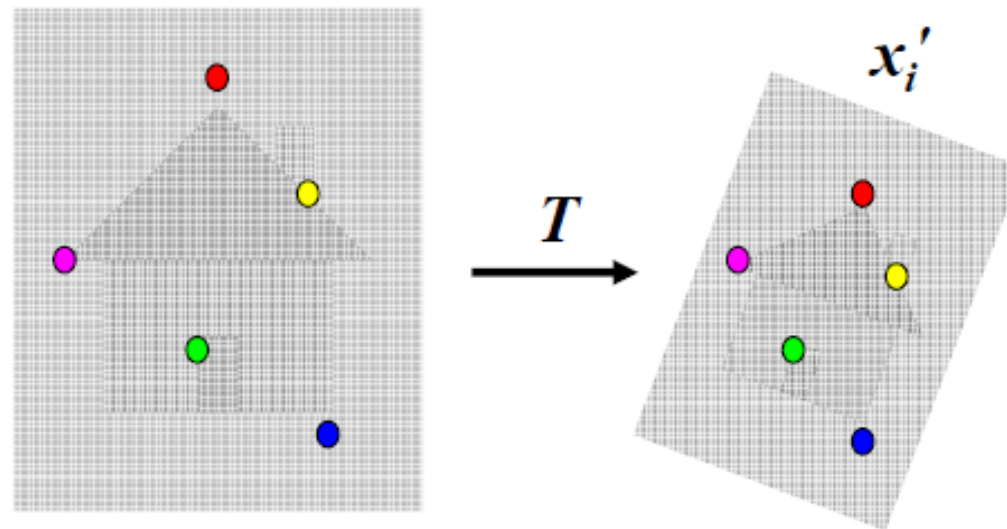


Fig. © Satya Mallick, et al. Homography examples using OpenCV (Python/C ++), LearnOpenCV, 2021, <https://learnopencv.com/homography-examples-using-opencv-python-c/>

Проблема поиска соответствий

- In alignment, we will fit the parameters of some transformation according to a set of matching feature pairs (“correspondences”).



Трансформации в анализе паттернов и форм

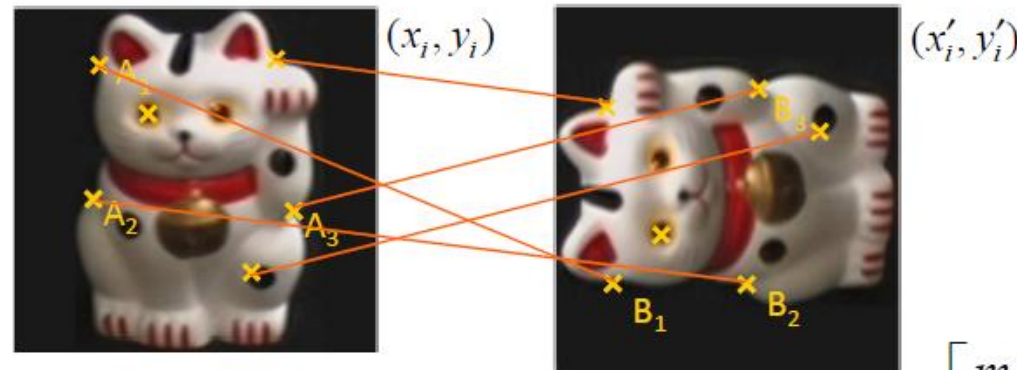
- Simple fitting procedure (linear least squares)
- Approximates viewpoint changes for roughly planar objects and roughly orthographic cameras
- Can be used to initialize fitting for more complex models



© Slide credit: Svetlana Lazebnik, David Lowe

Трансформации в анализе паттернов и форм

- Assuming we know the correspondences, how do we get the transformation?



$$\begin{bmatrix} x'_i \\ y'_i \end{bmatrix} = \begin{bmatrix} m_1 & m_2 \\ m_3 & m_4 \end{bmatrix} \begin{bmatrix} x_i \\ y_i \end{bmatrix} + \begin{bmatrix} t_1 \\ t_2 \end{bmatrix} \rightarrow \begin{bmatrix} m_1 \\ m_2 \\ m_3 \\ m_4 \\ t_1 \\ t_2 \end{bmatrix} = \begin{bmatrix} \end{bmatrix}$$

- Assuming we know the correspondences, how do we get the transformation?

$$\begin{bmatrix} x'_i \\ y'_i \end{bmatrix} = \begin{bmatrix} m_1 & m_2 \\ m_3 & m_4 \end{bmatrix} \begin{bmatrix} x_i \\ y_i \end{bmatrix} + \begin{bmatrix} t_1 \\ t_2 \end{bmatrix} \rightarrow \begin{bmatrix} \dots & \dots & \dots & \dots & \dots & \dots \\ x_i & y_i & 0 & 0 & 1 & 0 \\ 0 & 0 & x_i & y_i & 0 & 1 \\ \dots & \dots & \dots & \dots & \dots & \dots \end{bmatrix} \begin{bmatrix} m_1 \\ m_2 \\ m_3 \\ m_4 \\ t_1 \\ t_2 \end{bmatrix} = \begin{bmatrix} \dots \\ x'_i \\ y'_i \\ \dots \end{bmatrix}$$

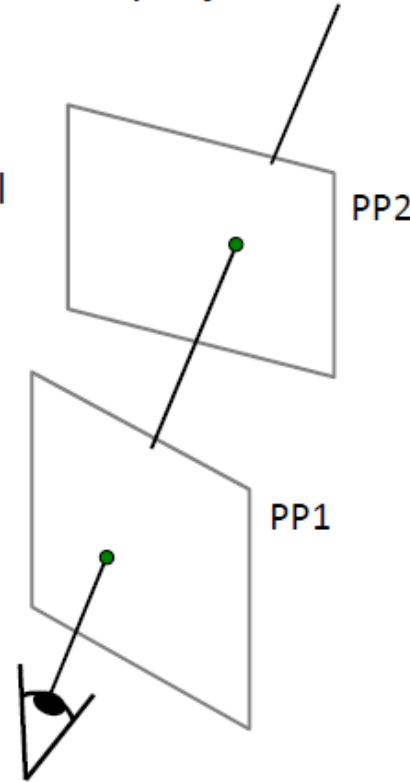
- How many matches (correspondence pairs) do we need to solve for the transformation parameters?
- Once we have solved for the parameters, how do we compute the coordinates of the corresponding point for (x_{new}, y_{new}) ?

Подгонка проективного преобразования

- A projective transform is a mapping between any two perspective projections with the same center of projection.
 - I.e. two planes in 3D along the same sight ray
- Properties
 - Rectangle should map to arbitrary quadrilateral
 - Parallel lines aren't
 - but must preserve straight lines
- This is called a **homography**

$$\begin{bmatrix} wx' \\ wy' \\ w \\ p' \end{bmatrix} = \begin{bmatrix} * & * & * \\ * & * & * \\ * & * & * \end{bmatrix} \begin{bmatrix} x \\ y \\ 1 \\ p \end{bmatrix}$$

H



Подгонка проективного преобразования

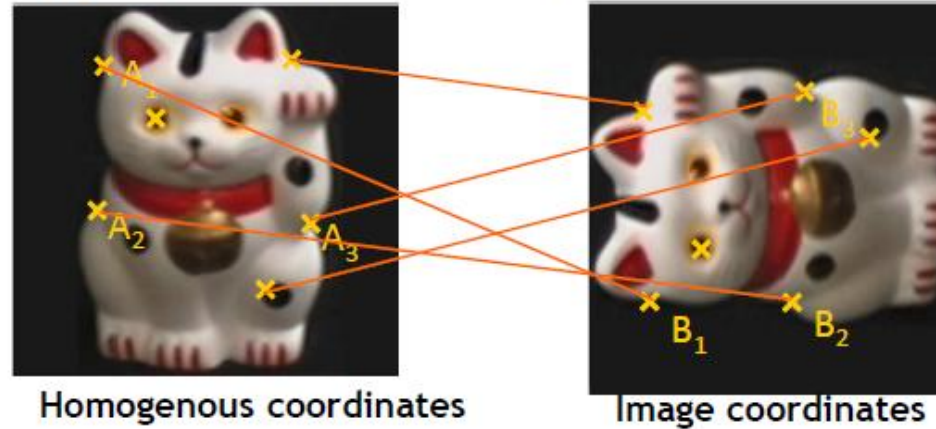
- A projective transform is a mapping between any two perspective projections with the same center of projection.
 - I.e. two planes in 3D along the same sight ray
- Properties
 - Rectangle should map to arbitrary quadrilateral
 - Parallel lines aren't
 - but must preserve straight lines
- This is called a **homography**

$$\begin{bmatrix} wx' \\ wy' \\ w \end{bmatrix}_p = \begin{bmatrix} h_{11} & h_{12} & h_{13} \\ h_{21} & h_{22} & h_{23} \\ h_{31} & h_{32} & \textcircled{1} \end{bmatrix}_H \begin{bmatrix} x \\ y \\ 1 \end{bmatrix}_p$$

Set scale factor to 1
⇒ 8 parameters left.

Проективное преобразование (Гомография)

- Estimating the transformation



$$\begin{aligned} \mathbf{X}_{A_1} &\leftrightarrow \mathbf{X}_{B_1} \\ \mathbf{X}_{A_2} &\leftrightarrow \mathbf{X}_{B_2} \\ \mathbf{X}_{A_3} &\leftrightarrow \mathbf{X}_{B_3} \\ &\vdots \end{aligned}$$

$$\begin{bmatrix} x' \\ y' \\ z' \end{bmatrix} = \begin{bmatrix} h_{11} & h_{12} & h_{13} \\ h_{21} & h_{22} & h_{23} \\ h_{31} & h_{32} & 1 \end{bmatrix} \cdot \begin{bmatrix} x \\ y \\ 1 \end{bmatrix}$$

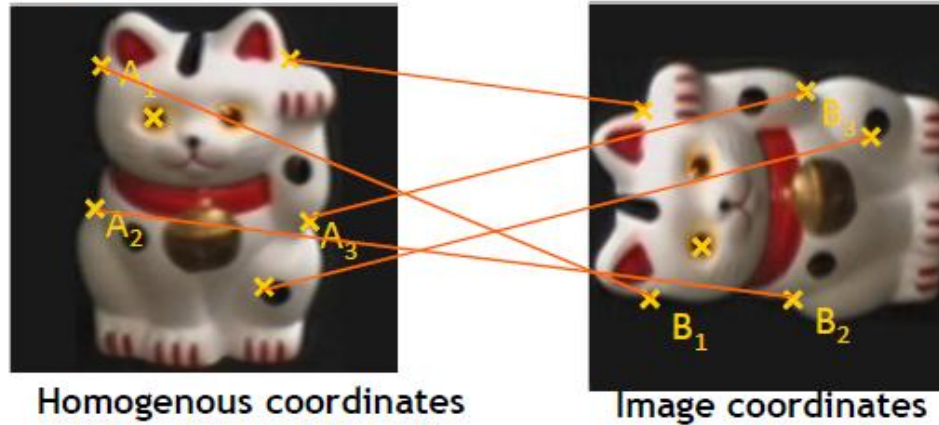
$$\begin{bmatrix} x'' \\ y'' \\ 1 \end{bmatrix} = \frac{1}{z'} \begin{bmatrix} x' \\ y' \\ z' \end{bmatrix}$$

Matrix notation

$$\begin{aligned} x' &= Hx \\ x'' &= \frac{1}{z'} x' \end{aligned}$$

Проективное преобразование (Гомография)

- Estimating the transformation



$$\begin{aligned} \mathbf{X}_{A_1} &\leftrightarrow \mathbf{X}_{B_1} \\ \mathbf{X}_{A_2} &\leftrightarrow \mathbf{X}_{B_2} \\ \mathbf{X}_{A_3} &\leftrightarrow \mathbf{X}_{B_3} \\ &\vdots \end{aligned}$$

$$\begin{bmatrix} x' \\ y' \\ z' \end{bmatrix} = \begin{bmatrix} h_{11} & h_{12} & h_{13} \\ h_{21} & h_{22} & h_{23} \\ h_{31} & h_{32} & 1 \end{bmatrix} \begin{bmatrix} x \\ y \\ 1 \end{bmatrix}$$
$$x_{A_1} = \frac{h_{11} x_{B_1} + h_{12} y_{B_1} + h_{13}}{h_{31} x_{B_1} + h_{32} y_{B_1} + 1}$$

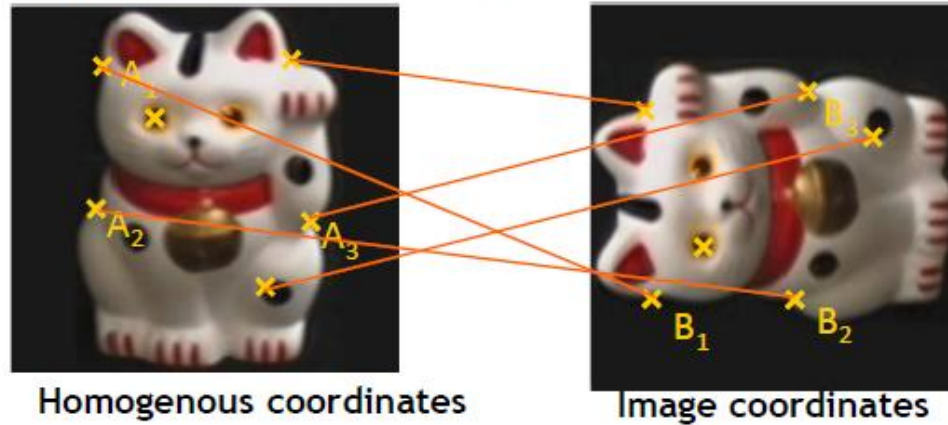
$$\begin{bmatrix} x'' \\ y'' \\ 1 \end{bmatrix} = \frac{1}{z'} \begin{bmatrix} x' \\ y' \\ z' \end{bmatrix}$$

Matrix notation

$$\begin{aligned} x' &= Hx \\ x'' &= \frac{1}{z'} x' \end{aligned}$$

Проективное преобразование (Гомография)

- Estimating the transformation

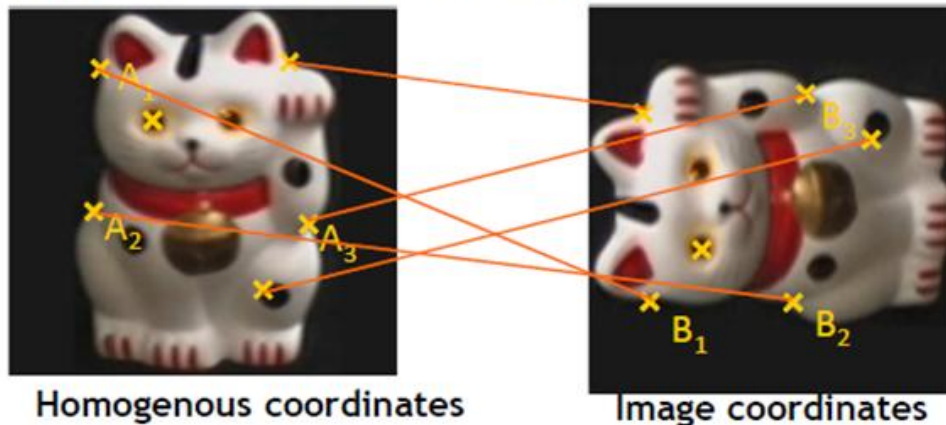


Matrix notation

$$\begin{aligned} \mathbf{X}_{A_1} &\leftrightarrow \mathbf{X}_{B_1} \\ \mathbf{X}_{A_2} &\leftrightarrow \mathbf{X}_{B_2} \\ \mathbf{X}_{A_3} &\leftrightarrow \mathbf{X}_{B_3} \\ &\vdots \end{aligned}$$
$$\begin{bmatrix} x' \\ y' \\ z' \end{bmatrix} = \begin{bmatrix} h_{11} & h_{12} & h_{13} \\ h_{21} & h_{22} & h_{23} \\ h_{31} & h_{32} & 1 \end{bmatrix} \begin{bmatrix} x \\ y \\ 1 \end{bmatrix}$$
$$x_{A_1} = \frac{h_{11} x_{B_1} + h_{12} y_{B_1} + h_{13}}{h_{31} x_{B_1} + h_{32} y_{B_1} + 1} \quad y_{A_1} = \frac{h_{21} x_{B_1} + h_{22} y_{B_1} + h_{23}}{h_{31} x_{B_1} + h_{32} y_{B_1} + 1}$$
$$\begin{bmatrix} x'' \\ y'' \\ 1 \end{bmatrix} = \begin{bmatrix} x' \\ y' \\ z' \end{bmatrix}$$
$$\begin{aligned} x' &= Hx \\ x'' &= \frac{1}{z'} x' \end{aligned}$$

Проективное преобразование (Гомография)

- Estimating the transformation



$$\mathbf{X}_{A_1} \leftrightarrow \mathbf{X}_{B_1}$$

$$\mathbf{X}_{A_2} \leftrightarrow \mathbf{X}_{B_2}$$

$$\mathbf{X}_{A_3} \leftrightarrow \mathbf{X}_{B_3}$$

$$\vdots$$

$$x_{A_1} = \frac{h_{11} x_{B_1} + h_{12} y_{B_1} + h_{13}}{h_{31} x_{B_1} + h_{32} y_{B_1} + 1}$$

$$y_{A_1} = \frac{h_{21} x_{B_1} + h_{22} y_{B_1} + h_{23}}{h_{31} x_{B_1} + h_{32} y_{B_1} + 1}$$

$$x_{A_1} h_{31} x_{B_1} + x_{A_1} h_{32} y_{B_1} + x_{A_1} = h_{11} x_{B_1} + h_{12} y_{B_1} + h_{13}$$

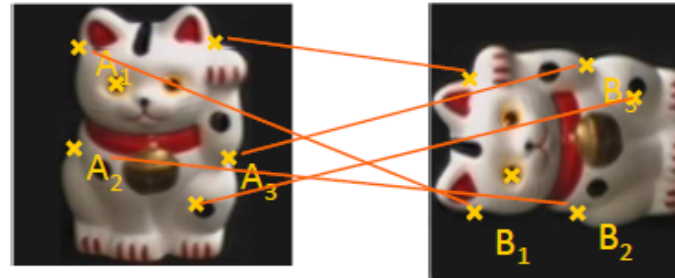
$$h_{11} x_{B_1} + h_{12} y_{B_1} + h_{13} - x_{A_1} h_{31} x_{B_1} - x_{A_1} h_{32} y_{B_1} - x_{A_1} = 0$$

$$h_{21} x_{B_1} + h_{22} y_{B_1} + h_{23} - y_{A_1} h_{31} x_{B_1} - y_{A_1} h_{32} y_{B_1} - y_{A_1} = 0$$

Проективное преобразование (Гомография)

- Estimating the transformation

$$\begin{aligned} h_{11} x_{B_1} + h_{12} y_{B_1} + h_{13} - x_{A_1} h_{31} x_{B_1} - x_{A_1} h_{32} y_{B_1} - x_{A_1} &= 0 \\ h_{21} x_{B_1} + h_{22} y_{B_1} + h_{23} - y_{A_1} h_{31} x_{B_1} - y_{A_1} h_{32} y_{B_1} - y_{A_1} &= 0 \end{aligned}$$



$$\mathbf{X}_{A_1} \leftrightarrow \mathbf{X}_{B_1}$$

$$\mathbf{X}_{A_2} \leftrightarrow \mathbf{X}_{B_2}$$

$$\mathbf{X}_{A_3} \leftrightarrow \mathbf{X}_{B_3}$$

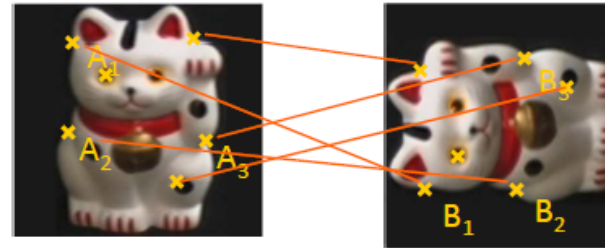
$\vdots$

$$\begin{bmatrix} x_{B_1} & y_{B_1} & 1 & 0 & 0 & 0 & -x_{A_1}x_{B_1} & -x_{A_1}y_{B_1} & -x_{A_1} \\ 0 & 0 & 0 & x_{B_1} & y_{B_1} & 1 & -y_{A_1}x_{B_1} & -y_{A_1}y_{B_1} & -y_{A_1} \\ \cdot & \cdot & \cdot & \cdot & \cdot & \cdot & \cdot & \cdot & \cdot \\ \cdot & \cdot & \cdot & \cdot & \cdot & \cdot & \cdot & \cdot & \cdot \\ \cdot & \cdot & \cdot & \cdot & \cdot & \cdot & \cdot & \cdot & \cdot \end{bmatrix} \cdot \begin{bmatrix} h_{11} \\ h_{12} \\ h_{13} \\ h_{21} \\ h_{22} \\ h_{23} \\ h_{31} \\ h_{32} \\ 1 \end{bmatrix} = \begin{bmatrix} 0 \\ 0 \\ \cdot \\ \cdot \\ \cdot \\ \cdot \\ \cdot \end{bmatrix}$$

$$Ah = 0$$

Проективное преобразование (Гомография)

- Estimating the transformation
- Solution:
 - Null-space vector of A
 - Corresponds to smallest eigenvector



$$\begin{aligned} \mathbf{x}_{A_1} &\leftrightarrow \mathbf{x}_{B_1} \\ \mathbf{x}_{A_2} &\leftrightarrow \mathbf{x}_{B_2} \\ \mathbf{x}_{A_3} &\leftrightarrow \mathbf{x}_{B_3} \\ &\vdots \end{aligned}$$

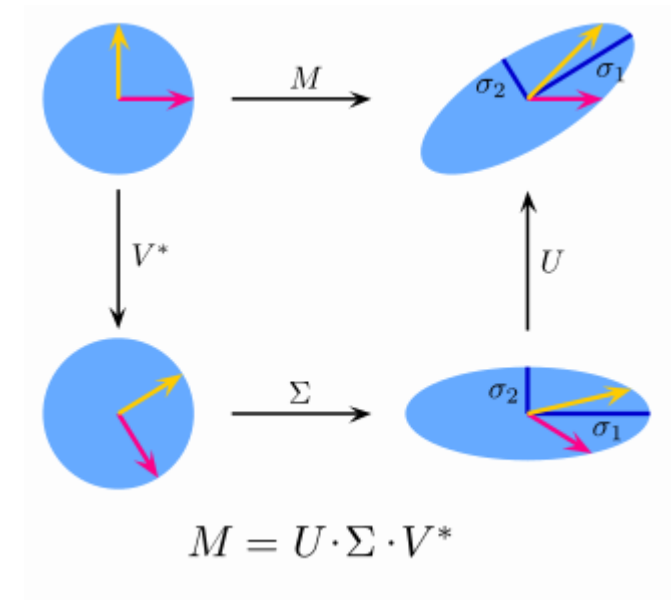
SVD

$$\mathbf{A} = \mathbf{U}\mathbf{D}\mathbf{V}^T = \mathbf{U} \begin{bmatrix} d_{11} & \cdots & d_{19} \\ \vdots & \ddots & \vdots \\ d_{91} & \cdots & d_{99} \end{bmatrix} \begin{bmatrix} v_{11} & \cdots & v_{19} \\ \vdots & \ddots & \vdots \\ v_{91} & \cdots & v_{99} \end{bmatrix}^T$$
$$\mathbf{h} = \frac{[v_{19}, \dots, v_{99}]}{v_{99}}$$

$Ah = 0$

Minimizes least square error

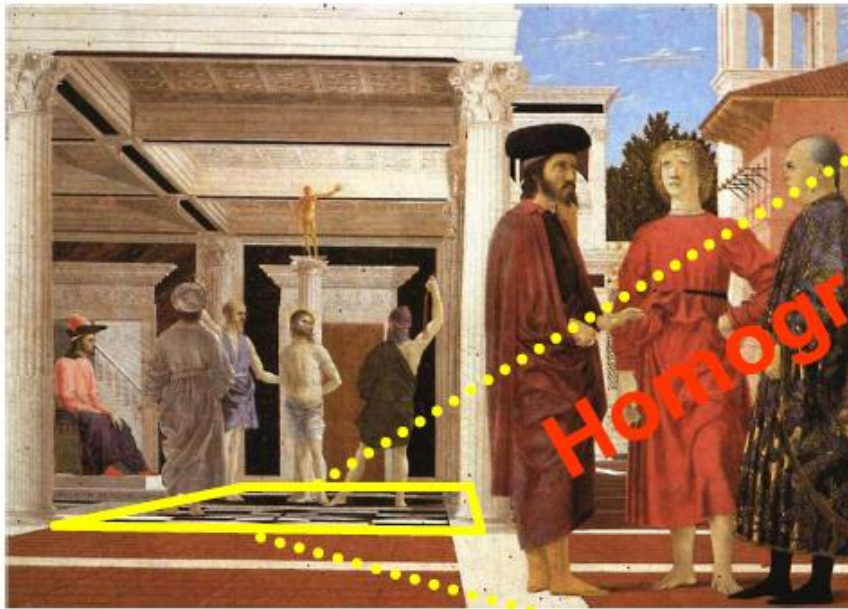
Если конфигурация точек хорошая (общее положение, 4 и более не коллинеарных точек), минимальное число σ_9 будет строго изолированным, и вектор h будет единственным (с точностью до знака). Если минимальное сингулярное число повторяется (например, $\sigma_8 = \sigma_9 = 0$). То столбцы v_8, v_9 можно вращать внутри этого подпространства как угодно, порождая бесконечно много разных векторов h . Но это случается строго в вырожденных случаях: когда точек меньше 4, или 3 из 4 точек лежат на одной прямой.



© Wiki: Singular value decomposition

Трансформации в анализе паттернов и форм

- What is the shape of the b/w floor pattern?



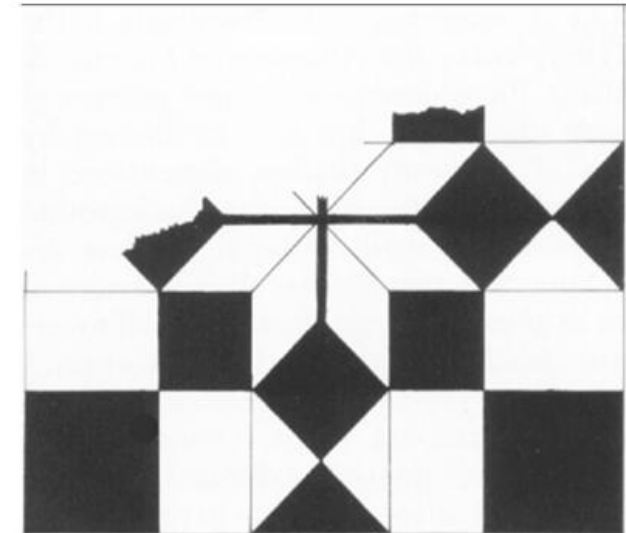
The floor (enlarged)



Homography



VS



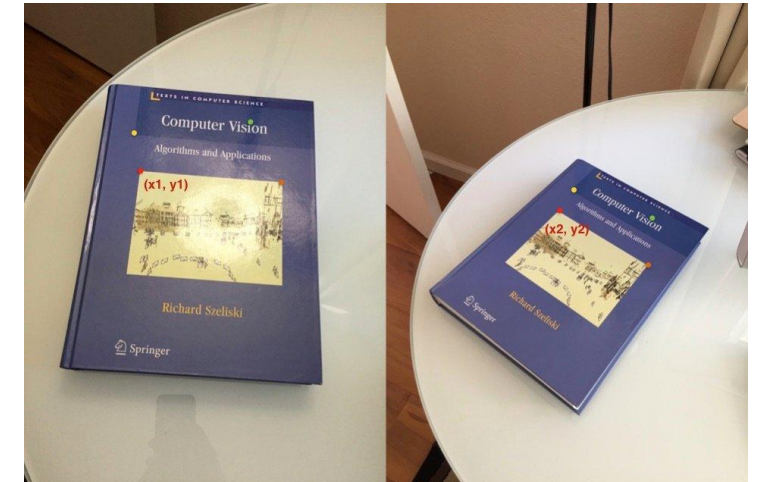
From Martin Kemp *The Science of Art*
(manual reconstruction)

Как вычислить гомографию?

Для вычисления гомографии между двумя изображениями необходимо знать как минимум четыре пары соответствующих точек. Если таких пар больше четырех — это даже лучше: OpenCV надежно вычислит гомографию, наилучшим образом соответствующую всем этим точкам.

```
1 #!/usr/bin/env python
2
3 import cv2
4 import numpy as np
5
6 if __name__ == '__main__':
7
8     # Read source image.
9     im_src = cv2.imread('book2.jpg')
10    # Four corners of the book in source image
11    pts_src = np.array([[141, 131], [480, 159], [493, 630], [64, 601]])
12
13    # Read destination image.
14    im_dst = cv2.imread('book1.jpg')
15    # Four corners of the book in destination image.
16    pts_dst = np.array([[318, 256], [534, 372], [316, 670], [73, 473]])
17
18    # Calculate Homography
19    h, status = cv2.findHomography(pts_src, pts_dst)
20
21    # Warp source image to destination based on homography
22    im_out = cv2.warpPerspective(im_src, h, (im_dst.shape[1], im_dst.shape[0]))
23
24    # Display images
25    cv2.imshow("Source Image", im_src)
26    cv2.imshow("Destination Image", im_dst)
27    cv2.imshow("Warped Source Image", im_out)
28
29    cv2.waitKey(0)
```

Обычно соответствия между точками находятся автоматически путем сопоставления таких признаков, как SIFT или SURF, на разных изображениях, но в этой статье мы просто будем отмечать точки вручную.



Применяя гомографию к левому изображению

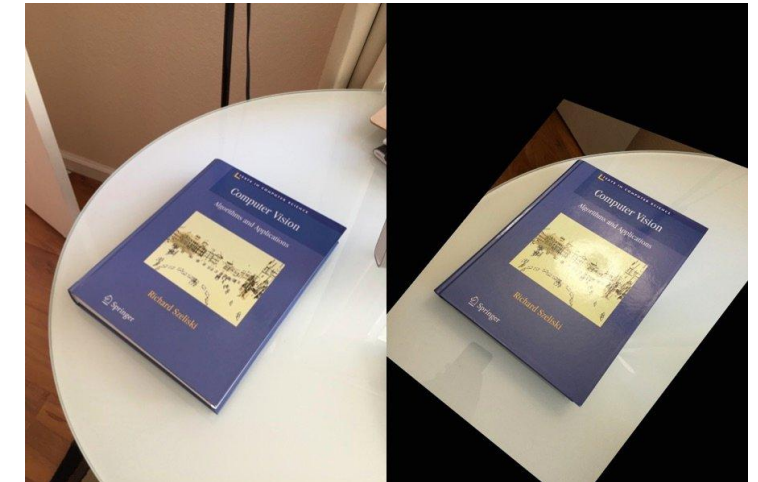


Fig. © Satya Mallick, et al. Homography examples using OpenCV (Python/C ++), LearnOpenCV, 2021, <https://learnopencv.com/homography-examples-using-opencv-python-c/>

Аффинное (6 DoF)

- три пары точек, система всегда решается, если точки не на прямой
- параллельность сохранена — удобно для сеток и текста
- стоит `warpAffine` ≈ 3 мс; описывает поворот, масштаб, скос
- **не описывает** плакат под углом: дальний край не сжимается

Проективное (8 DoF)

- четыре пары точек; работает только для плоскости или чистого поворота камеры
- параллельность потеряна, прямые сохранены; растяжение неравномерно
- стоит `warpPerspective` ≈ 3.5 мс: деление на w в каждом пикселе
- **ломается** на дисторсии, на не-плоскости, на горизонте в кадре

- Теперь умеем: записать любое преобразование плоскости матрицей 3×3 в однородных координатах, назвать DoF, число пар точек и инварианты каждой степени; собрать композицию в правильном порядке
- Знаем цену: аффинное — 3 пары и параллельность; гомография — 4 пары, законна только для плоскости; клик ± 2 px $\rightarrow$ ошибка 4 px
- Чего пока не умеем: **исполнить** преобразование — координаты после H дробные, а пиксели стоят на целой сетке. Это вторая часть

Демо 1.1 — поворот с сохранением кадра

python

```
1 import cv2, numpy as np
2 from cvcourse import io as cio
3 img = cio.imread("demo/iu-building.jpg")          # 1920 x 1279
4 h, w = img.shape[:2]
5 M = cv2.getRotationMatrix2D((w / 2, h / 2), 30, 1.0) # 2 x 3: T(c)·R·T(-c)
6 cos, sin = abs(M[0, 0]), abs(M[0, 1])
7 nw, nh = round(w * cos + h * sin), round(w * sin + h * cos)
8 M[0, 2] += nw / 2 - w / 2                        # новый центр холста
9 M[1, 2] += nh / 2 - h / 2
10 out = cv2.warpAffine(img, M, (nw, nh), borderValue=(255, 255, 255))
11 print("кадр %d x %d → холст %d x %d" % (w, h, nw, nh))
12 print("M =", np.round(M, 2).tolist())
```

кадр $1920 \times 1279 \rightarrow$ холст 2302×2068
 $M = [[0.87, 0.5, -0.13], [-0.5, 0.87, 960.18]]$

Вызов

`cv2.getRotationMatrix2D((cx, cy), 30, 1.0)`

`w·cos + h·sin, w·sin + h·cos`

`cv2.warpAffine(img, M, (nw, nh), borderValue=...)`

Что делает

матрица 2×3 поворота на 30° против часовой вокруг центра, масштаб 1

габариты повернутого прямоугольника — размер холста без потерь

применяет M обратным отображением; dsize — (ширина, высота)

Демо 1.2 — 4 точки → фронтальный вид

python

```
1 img = cio.imread("demo/poster-angle.jpg")
2 src = np.float32([[560, 330], [1250, 220], [1290, 1010], [600, 840]])
3 W, Hh = 640, 905 # tl, tr, br, bl; A4 1:1.414
4 dst = np.float32([[0, 0], [W - 1, 0], [W - 1, Hh - 1], [0, Hh - 1]])
5 H = cv2.getPerspectiveTransform(src, dst) # 3 × 3, кадр → лист
6 front = cv2.warpPerspective(img, H, (W, Hh), flags=cv2.INTER_LINEAR)
7 gt = cio.imread("demo/poster-gt.png") # плакат «в лоб»
8 ink = lambda im: cv2.cvtColor(im, cv2.COLOR_BGR2GRAY) < 128 # чернила
9 p0 = cv2.perspectiveTransform(src[None, :1], H)[0, 0].round(4)
10 print("H =", np.round(H, 4).tolist())
11 print("угол (560, 330) →", p0)
12 print("совпало: %.1f %% пикселей" % (100 * (ink(front) == ink(gt)).mean()))
```

$H = [[2.4241, -0.1901, -1294.8], [0.4645, 2.914, -1221.7], [0.0014, -0.0002, 1.0]]$
угол (560, 330) → [0. 0.] · совпало: 99.1 %
пикселей

Вызов

Что делает

`cv2.getPerspectiveTransform(src, dst)`

гомография 3×3 по четырём парам точек — порядок точек в `src` и `dst` должен совпадать

`cv2.warpPerspective(img, H, (W, Hh), flags=...)`

для каждого пикселя листа берёт $H^{-1} \cdot (x', y', 1)$, делит на w , интерполирует

`cv2.perspectiveTransform(pts, H)`

применяет H к точкам (форма $(1, N, 2)$), с делением на w — проверка соответствий

Демо 1.3 — что сохраняется, а что нет

python

```
1 pts = np.float32([[20, 870], [120, 870], [320, 870], [620, 870]]) # линейка
2 q = cv2.perspectiveTransform(pts, np.linalg.inv(H))[0] # те же точки в кадре
3 p = pts[0]
4 d = lambda a, b: float(np.linalg.norm(a - b))
5 cr = lambda u: d(u[0], u[2]) * d(u[1], u[3]) / (d(u[1], u[2]) * d(u[0], u[3]))
6 print("AB/BC: лист %.3f, кадр %.3f" % (d(p[0], p[1]) / d(p[1], p[2]),
7                                     d(q[0], q[1]) / d(q[1], q[2])))
8 print("двойное отношение: лист %.4f, кадр %.4f" % (cr(p), cr(q)))
9 A = np.float32([[1.2, 0.3, 10], [0.1, 0.8, 5]]) # аффинное 2 x 3
10 sq = np.float32([[0, 0], [100, 0], [100, 100], [0, 100]])
11 print("det A = %.2f; площадь 10000 → %.0f"
12       % (np.linalg.det(A[:, :2]), cv2.contourArea(cv2.transform(sq, A))))
```

AB/BC: лист 0.500, кадр 0.416 · двойное
отношение: лист 1.2500, кадр 1.2500 · $\det A =$
0.93; площадь 10000 → 9300

Вызов

`np.linalg.inv(H)`

$(AC \cdot BD) / (BC \cdot AD)$

`cv2.transform(sq, A) · cv2.contourArea`

Что делает

обратная гомография «лист → кадр»; `perspectiveTransform`
делит на w сам

двойное отношение четырёх точек прямой — инвариант
проективного преобразования

аффинное к точкам (без деления) и площадь многоугольника —
проверка $\det A$

Демо 1.4 — цена варпа

```
1 import timeit
2 ms = lambda fn: 1e3 * min(timeit.repeat(fn, number=1, repeat=5))
3 R = cv2.getRotationMatrix2D((w / 2, h / 2), 10, 1.0)
4 P = np.vstack([R, [0, 0, 1]]); P[2, :2] = (1e-5, 2e-5) # чуть перспективы
5 wp = lambda **kw: cv2.warpPerspective(img, P, (w, h), **kw)
6 for name, fn in (("warpAffine", lambda: cv2.warpAffine(img, R, (w, h))),
7                 ("warpPerspective", wp),
8                 ("+ CUBIC", lambda: wp(flags=cv2.INTER_CUBIC)),
9                 ("+ LANCZOS4", lambda: wp(flags=cv2.INTER_LANCZOS4))):
10     print("%-16s %5.1f мс на 1920 × 1279" % (name, ms(fn)))
```

python

warpAffine 3.0 мс · warpPerspective 3.5 мс · +
CUBIC 7.7 мс · + LANCZOS4 19.2 мс на 1920 ×
1279

Вызов

`timeit.repeat(fn, number=1, repeat=5)`

`P[2, :2] = (1e-5, 2e-5)`

`flags=cv2.INTER_CUBIC / INTER_LANCZOS4`

Что делает

пять прогонов по одному вызову; берём минимум — он ближе всего к «чистому» времени

перспективные члены h_{31} , h_{32} : матрица перестаёт быть аффинной, появляется деление на w

4 × 4 и 8 × 8 соседей на пиксель вместо 2 × 2 — цена растёт в 2 и 5–6 раз

- **Вопрос 1.** Фотография фасада здания с двумя стенами под углом 90° . Выпрямит ли её одна гомография по четырём точкам — да или нет, и почему?
- **Вопрос 2.** Матрицу поворота на 30° применили к кадру дважды подряд, `warpAffine` за `warpAffine`. Получится ли то же, что один поворот на 60° — верно или неверно?

Разбор — во второй части, измерением

B1: нет — гомография одна на плоскость; две стены — две матрицы, граница между ними «сломается»

B2: неверно по пикселям — каждый варп интерполирует заново, дважды размытая картинка; композиция $R_{60} = R_{30} \cdot R_{30}$ и один варп — правильно

2

Ресемплинг: как исполнить преобразование

- Прямой и обратный варп; интерполяция и её ядра; алиасинг при уменьшении; гетар — любая карта; границы, параметры, вырожденные случаи; четыре демо

- **Суть проблемы: Математика против пикселей:**

- Исходная картинка (src) и финальная (dst) состоят из строго целых пикселей (например, координата может быть только 312 или 313).
- Но когда мы применяем матрицу трансформации, формулы выдают **дробные координаты** — например, пиксель должен переехать в точку (417.3, 96.8). В цифровом изображении физически нет дробного пикселя. Значение цвета в этой точке нужно рассчитать — это и есть **интерполяция**.

- **Что такое ресемплинг:**

- **Ресемплинг** — это математический пересчет непрерывного сигнала (картинки) с одной дискретной сетки на другую.
- Матрица геометрии лишь указывает, **куда** на холсте должен попасть пиксель, а алгоритм ресемплинга решает, **какой именно цвет** туда записать (взять ли цвет ближайшего соседа или усреднить несколько соседних пикселей).

- **Критерии хорошего алгоритма:**

- **Покрытие:** Чтобы на итоговой картинке не осталось пустых «дыр» и черных пикселей.
- **Качество (PSNR):** Насколько финальная картинка близка к идеальному математическому эталону (отсутствие размытия или «лесенок» на гранях).
- **Время:** Алгоритм должен успевать обрабатывать кадр за единицы миллисекунд.

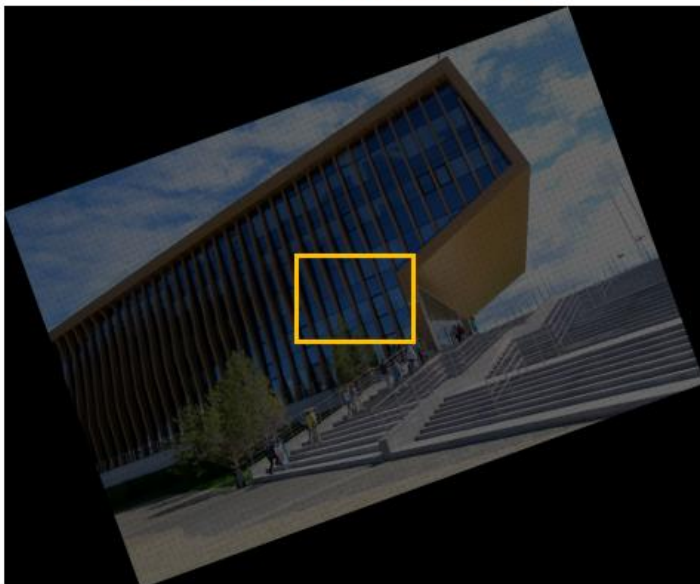
Наивное решение: прямой варп (warping)

- Проблема «Дыр» (Пропуски пикселей):
 - **Что происходит:** Когда мы переносим пиксели на новый холст и округляем их координаты, математика «промахивается» мимо целых клеток.
 - **Масштаб проблемы:** При обычном повороте на 20° внутри объекта останется **8% пустых (черных) пикселей**. А если поворот совместить с увеличением картинки в 1.6 раза, то **61% пикселей вообще не получают никакого значения** — изображение превратится в решето.
- Проблема «Столкновений» (Потеря данных):
 - **Что происходит:** При уменьшении изображения (например, в 2 раза) возникает обратная ситуация — сразу **4 исходных пикселя претендуют на то, чтобы сесть в одну и ту же клетку** новой картинки. Алгоритму непонятно, «кто победит» и чей цвет в итоге записывать, из-за чего теряются детали.
- Низкая скорость (Вычислительная цена):
 - Если реализовывать такой цикл перебора пикселей вручную на чистом Python, обработка кадра займет **сотни миллисекунд** (вместо нормальных 3–4 мс). При этом нормальную интерполяцию (сглаживание) при таком подходе сделать технически невозможно — приходится использовать только грубое округление.

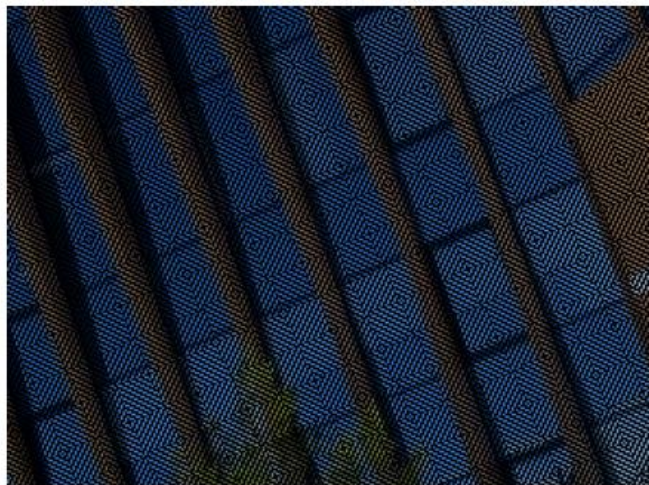
Прямой перенос пикселей из источника в приемник (прямой варпинг) не работает на практике, поскольку отвечает на вопрос «куда летит пиксель», а нужно отвечать на вопрос «откуда взять пиксель». Чтобы этого избежать, во всех библиотеках компьютерного зрения (включая OpenCV) используют обратный варп (backward warping), когда компьютер идет по пикселям готового холста и ищет их корни в исходном изображении.

Прямой против обратного варпа

прямой варп (forward): $\text{src} \rightarrow \text{round}(M \cdot x)$
дыр в области объекта: 61 %



фрагмент:
сетка непокрытых пикселей



обратный варп (inverse): $\text{dst} \leftarrow \text{src}(M^{-1} \cdot x')$
каждый пиксель результата получает значение



Прямой варп поворота с увеличением $\times 1.6$: 61 % пикселей внутри объекта не получили значения — сетка дыр по всей картинке. Обратный варп — для каждого пикселя результата ищем точку источника через M^{-1} и интерполируем — покрывает всё без единой дыры. Так работают `warpAffine`, `warpPerspective` и `remap`

При прямом варпе алгоритм перебирает пиксели **исходного изображения** и идет по «старой» упорядоченной сетке, «выстреливая» пикселями в новый холст. Новые координаты (x_{float} , y_{float}) могут получиться дробными (например, (45.2, 12.8)). Если мы не округлим их до целых (45, 13), мы просто не сможем записать цвет в матрицу `dst_image`, потому что ячейки памяти дискретны. Мы не знаем, какие еще пиксели прилетят в соседние клетки (45, 12) или (46, 13), чтобы делать интерполяцию и прилетят ли вообще. В результате при увеличении (scaling) пиксели разлетаются далеко друг от друга, оставляя 61% пустых дыр, как на фрагменте в центре.

© фото — здание Университета Иннополис; собственная иллюстрация

```
1 1 H_inv = inv(H) # результат → источник
2 2 для каждого пикселя (x', y') результата:
3 3     (u, v, w) = H_inv · (x', y', 1) # однородные координаты
4 4     x = u / w, y = v / w # деление – перспектива
5 5     если (x, y) внутри источника:
6 6         dst[y', x'] = interp(src, x, y) # NEAREST / LINEAR / CUBIC / LANCZOS4
7 7     иначе: dst[y', x'] = граница # borderMode, borderValue
```

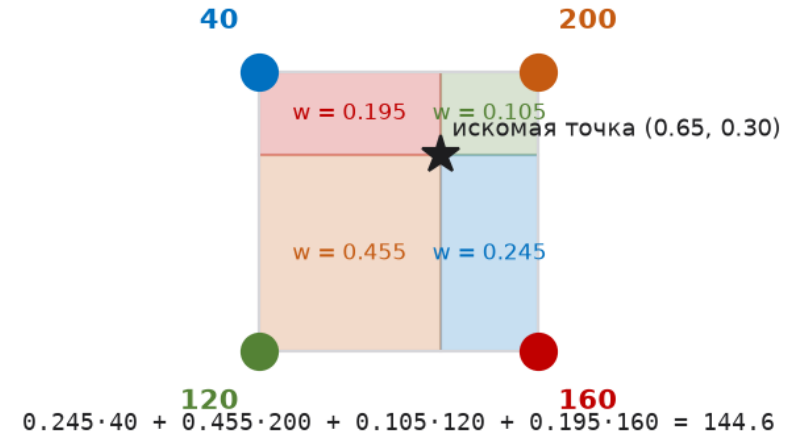
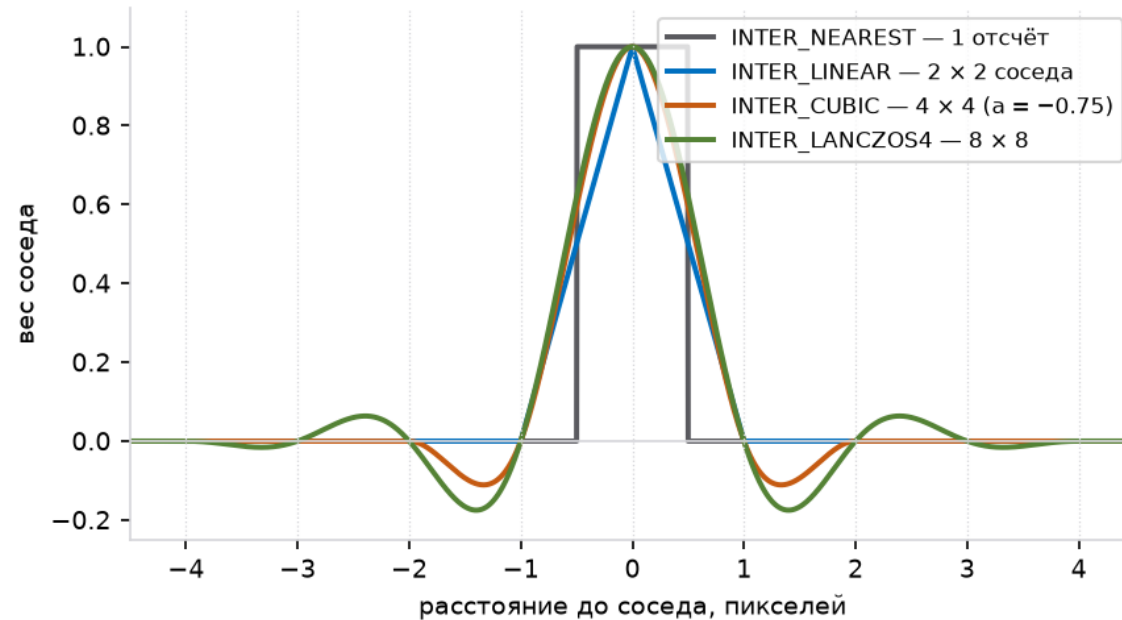
text

- Шаг 3 для аффинного — без деления: поэтому `warpAffine` быстрее `warpPerspective`
- Шаг 6 — единственное место, где придумываются значения; всё остальное — арифметика координат
- `remap` — то же самое, но шаги 1–4 заменены готовыми картами `map_x`, `map_y`

При обратном варпе алгоритм делает всё ровно наоборот: он перебирает пиксели **финального (нового) холста** (`dst`). Почему здесь интерполяция работает идеально? Потому что дробная координата (`x_float`, `y_float`) образуется внутри исходного изображения (`src`), где все соседние пиксели нам уже заведомо известны и лежат на своих местах. Если обратная матрица говорит, что для целого пикселя (10, 10) на новом холсте нужен цвет из координаты (312.6, 88.3) из `src`, алгоритм берет четыре реальных соседних пикселя (см. следующий слайд), и, считая их веса (через площади прямоугольников) находит идеальный сглаженный цвет.

ядро = вес соседа в зависимости от расстояния; отрицательные лепестки → резкость и ореолы

билинейная: вес соседа = площадь прямоугольника между точкой и противоположным соседом



Ядро интерполяции — вес соседа как функция расстояния: ближайший сосед берёт один отсчёт, билинейная — 2 × 2 с весами-площадями, бикубическая (α = −0.75) — 4 × 4, Lanczos4 — 8 × 8. Отрицательные лепестки дают резкость и ореолы у контрастных границ. Справа: билинейная в числах — $0.245 \cdot 40 + 0.455 \cdot 200 + 0.105 \cdot 120 + 0.195 \cdot 160 = 144.6$

© собственная иллюстрация

Четыре флага при увеличении

оригинал 640 × 480 (фрагмент)	INTER_NEAREST — текст ×4 PSNR 17.2 дБ · 0.06 мс	INTER_LINEAR — текст ×4 PSNR 17.3 дБ · 0.05 мс	INTER_CUBIC — текст ×4 PSNR 17.9 дБ · 0.14 мс	INTER_LANCZOS4 — текст ×4 PSNR 17.9 дБ · 0.45 мс
-------------------------------	--	---	--	---

Introduction to Computer Vision	Introduction to Computer Vision	Introduction to Computer Vision	Introduction to Computer Vision	Introduction to Computer Vision
Innopolis University, Fall 2026	Innopolis University, Fall 2026	Innopolis University, Fall 2026	Innopolis University, Fall 2026	Innopolis University, Fall 2026

текст, уменьшенный ×4
(INTER_AREA): 160 × 120

Introduction to Computer Vision
Innopolis University, Fall 2026
Lecture 4: transformations and
resampling. Homogeneous coords:
 $x' \sim Hx$, 8 DoF, 4 point pairs.
Inverse warping: for every output
pixel look up the source, then
interpolate: nearest / bilinear.

фото ×8 (20 × 15 → 160 × 120):
19.9 дБ

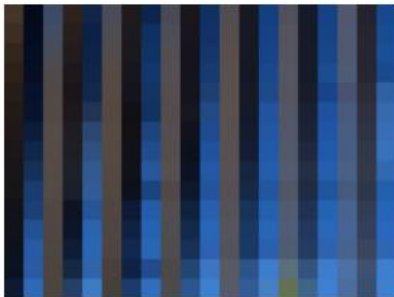


фото ×8 (20 × 15 → 160 × 120):
20.6 дБ



фото ×8 (20 × 15 → 160 × 120):
21.8 дБ

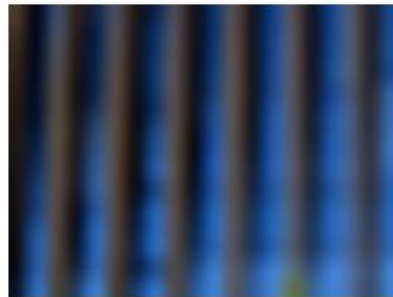


фото ×8 (20 × 15 → 160 × 120):
22.1 дБ



Текст, уменьшенный ×4 и увеличенный обратно: NEAREST 17.2 дБ, LINEAR 17.3, CUBIC и LANCZOS4 — 17.9; при ×8 все четыре дают 15 дБ — если тонкая линия текста уничтожена уменьшением, то её никакая интерполяция не вернёт.

Фото ×8: 19.9 → 20.6 → 21.8 → 22.1 дБ — на гладких перепадах широкое ядро выигрывает 2 дБ

© фото — здание Университета Иннополис; собственная иллюстрация

Флаг интерполяции: когда какой

Флаг	Соседей	Когда брать	Цена и подвох
INTER_NEAREST	1	маски, метки классов, глубина — новые значения не изобретаются	лесенки; при LINEAR маска получает 136 «серых» уровней
INTER_LINEAR	2 × 2	по умолчанию: варп в реальном времени, умеренное увеличение	мыло на большом увеличении; при уменьшении > 1.5× — алиасинг
INTER_CUBIC	4 × 4	увеличение фотографий, когда есть 2–3× по времени	ореолы у резких границ; +2 дБ на фото ×8
INTER_LANCZOS4	8 × 8	максимум качества офлайн	5–8× по времени; выигрыш над CUBIC — 0.3 дБ
INTER_AREA	окно	только `resize` при уменьшении	в warpAffine/warpPerspective /remap молча заменяется на LINEAR

Уменьшение: алиасинг и его лечение

warpAffine $\times \frac{1}{4}$, INTER_LINEAR: 27.9 дБ
алиасинг — 2×2 соседа из 16



Если уменьшить картинку с частыми полосами (например, окна) в 4 раза, используя warpAffine с INTER_LINEAR, линии пойдут волнами или ступеньками (алиасинг)

GaussianBlur $\sigma = 1.5 \rightarrow$ тот же варп:
36.7 дБ



Способ лечения №1: Размытие перед уменьшением с помощью фильтра Гаусса. Размытие стирает мелкие детали, которые не могут поместиться в уменьшенную картинку.

cv2.resize INTER_AREA: 37.2 дБ
среднее по 4×4

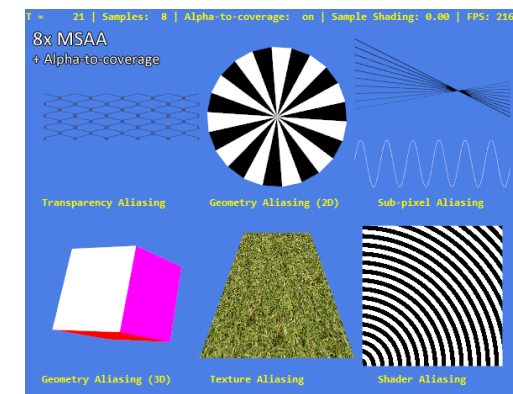
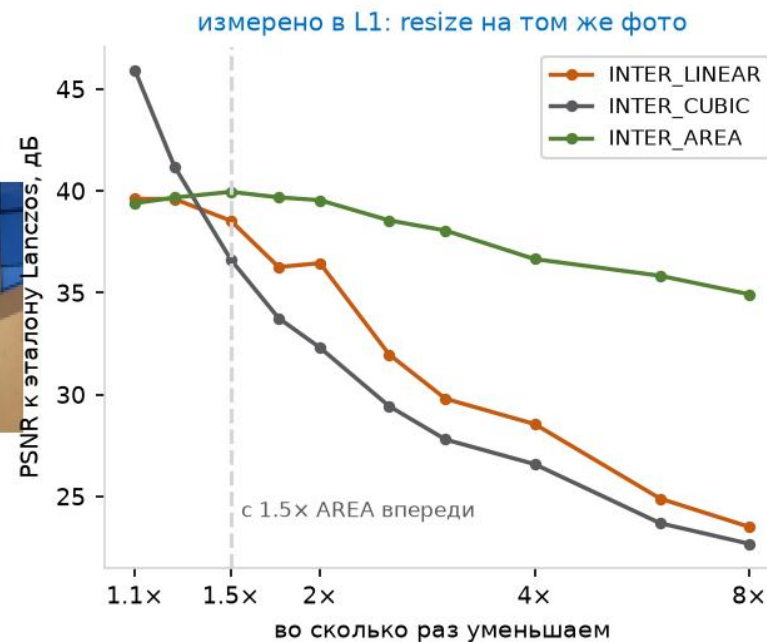


Способ лечения №2: вместо размытие + уменьшение используется cv2.resize с флагом INTER_AREA, усредняющая цвет в блоке.

warpAffine $\times \frac{1}{4}$ с INTER_LINEAR берёт 2×2 соседа из 16 — алиасинг, 27.9 дБ к эталону Lanczos. Размыть заранее GaussianBlur $\sigma = 1.5$ и тот же варп — 36.7 дБ. cv2.resize INTER_AREA усредняет окно 4×4 — 37.2 дБ. Справа — измерение L1: AREA впереди с $1.5\times$

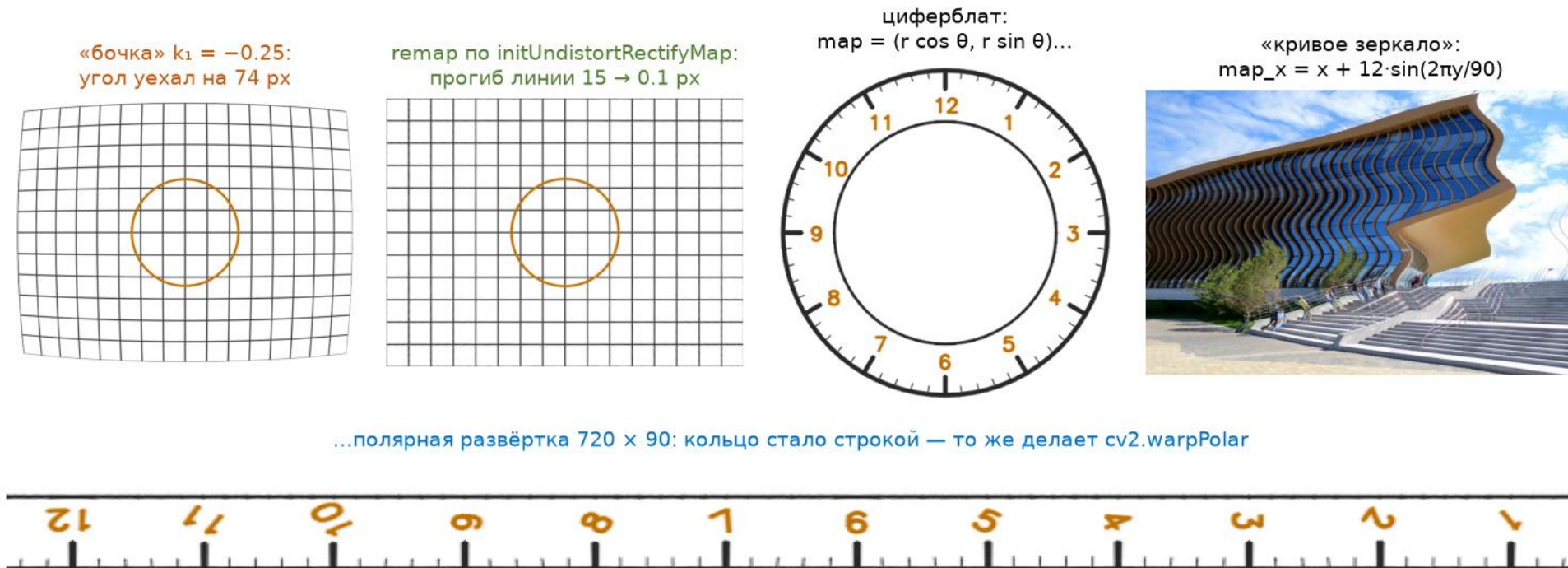
Алиасинг (Aliasing) - **Математическая ошибка** оцифровки, когда частота деталей объекта выше, чем разрешающая способность матрицы (нарушается теорема Котельникова/Найквиста). Выглядят, как пиксельные «лесенки»/волны.

© фото — здание Университета Иннополис; собственная иллюстрация



© fig.: [Алгоритмы антиалиасинга в реальном времени](#), Habr, 2017

remap: три карты



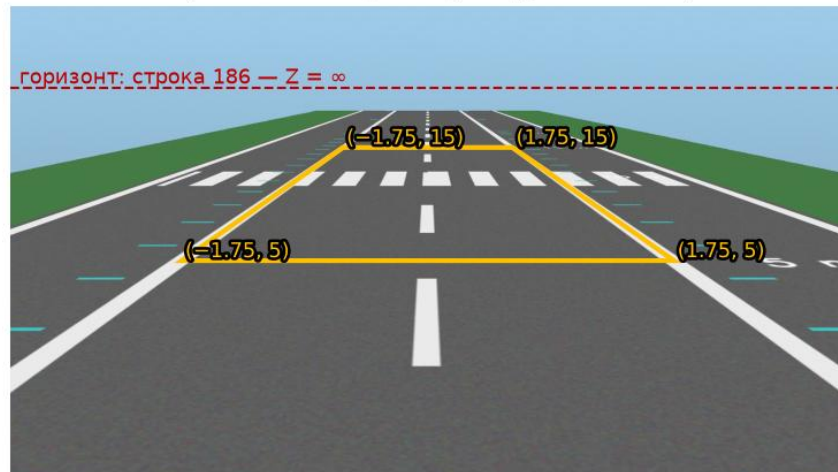
Слева: «бочка» $k_1 = -0.25$ уносит угол кадра на 74 px, прогиб линии сетки — 15 px; remap по карте `initUndistortRectifyMap` возвращает прогиб к 0.1 px — прямые снова прямые. В центре и внизу: циферблат по карте $(r \cos \theta, r \sin \theta)$ разворачивается в строку 720 $\times$ 90 — кольцо стало текстом. Справа: «кривое зеркало» — синусоидальная карта

В функцию `cv2.remap` передаются специальные «карты» (координатные сетки), которые указывают индивидуально для каждого пикселя, откуда его нужно взять

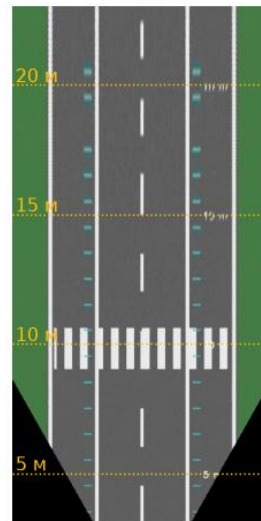
© фото — здание Университета Иннополис; собственная иллюстрация

Bird's-eye view: метры и их цена

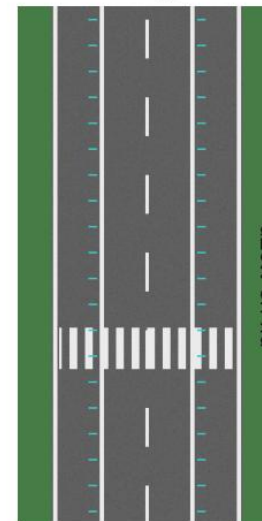
кадр 1920 × 1080: f_x 1663, $h = 1.2$ м, наклон 12°
четыре точки на асфальте, координаты в метрах



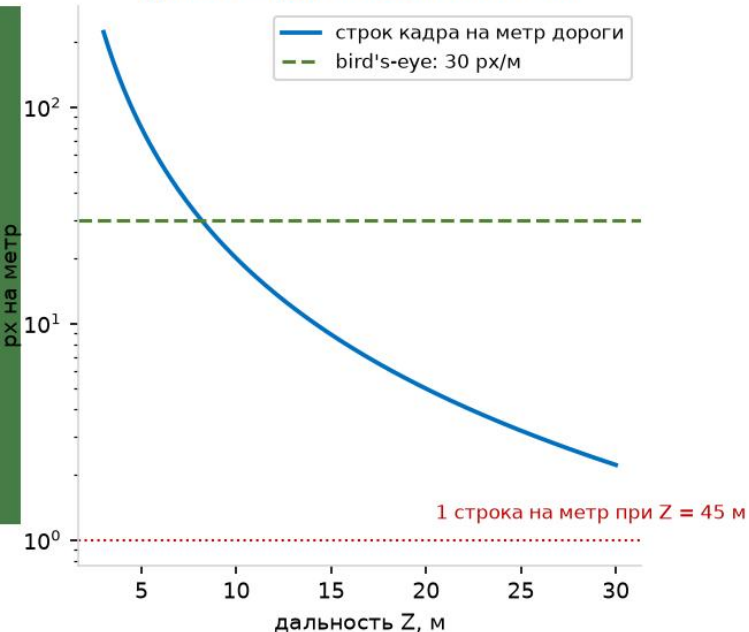
bird's-eye view
30 px/м



эталон:
текстура



ближе 8 м bev уменьшает (алиасинг),
дальше — растягивает: на 20 м × 6

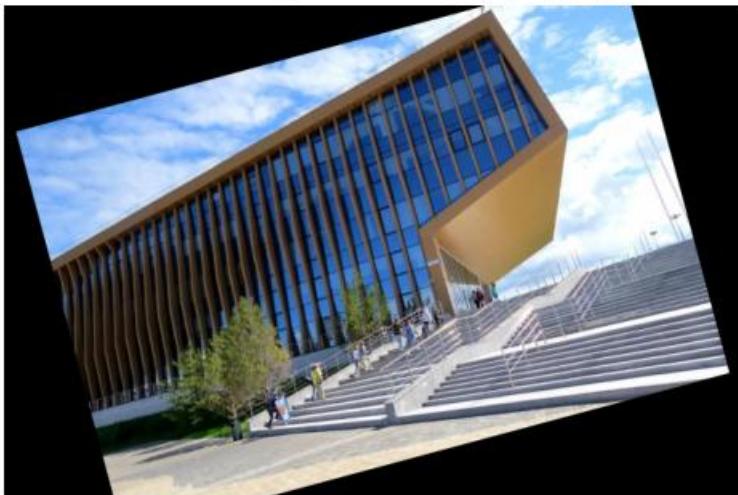


Камера робота: f_x 1663, высота 1.2 м, наклон 12° . Четыре точки на асфальте с известными метрами → вид сверху 30 px/м: ширина полосы измеряется точно — 3.5 м и на 8 м, и на 20 м. Но плотность строк кадра падает как $1/Z^2$: 80 строк на метр в 5 м, 5 строк — в 20 м; bird's-eye растягивает их в 6 раз, фронт штриха разметки размывается с 1.5 до 12 строк. Горизонт — строка 186 — уходит в бесконечность

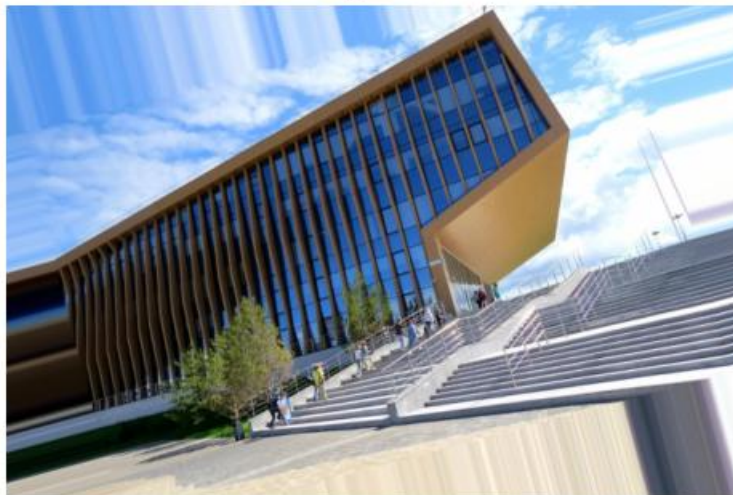
Гомография позволяет идеально выпрямить дорогу в метрическую сетку для автопилота, но из-за оптики и геометрии камера «видит» дальние участки слишком плохо, поэтому вид сверху на расстоянии от 20 метров и дальше математически становится очень размытым и неточным.

© синтетическая сцена по модели камеры L2; собственная иллюстрация

BORDER_CONSTANT (0, 0, 0)



BORDER_REPLICATE



BORDER_REFLECT_101



Один поворот, три режима: BORDER_CONSTANT заполняет чёрным (или `borderValue`), BORDER_REPLICATE тянет крайние пиксели полосами, BORDER_REFLECT_101 отражает без повторения края. Для трекара и склейки — CONSTANT с маской; для фильтров и пирамид (L5–L6) — REFLECT_101

При трансформациях всегда вылезают «пустые» углы, и выбор правильного флага (`borderMode`) определяет, зальются они черным цветом (для масок) или зеркально продолжат картинку (для фильтров).

© фото — здание Университета Иннополис; собственная иллюстрация

Параметры трёх функций варпа

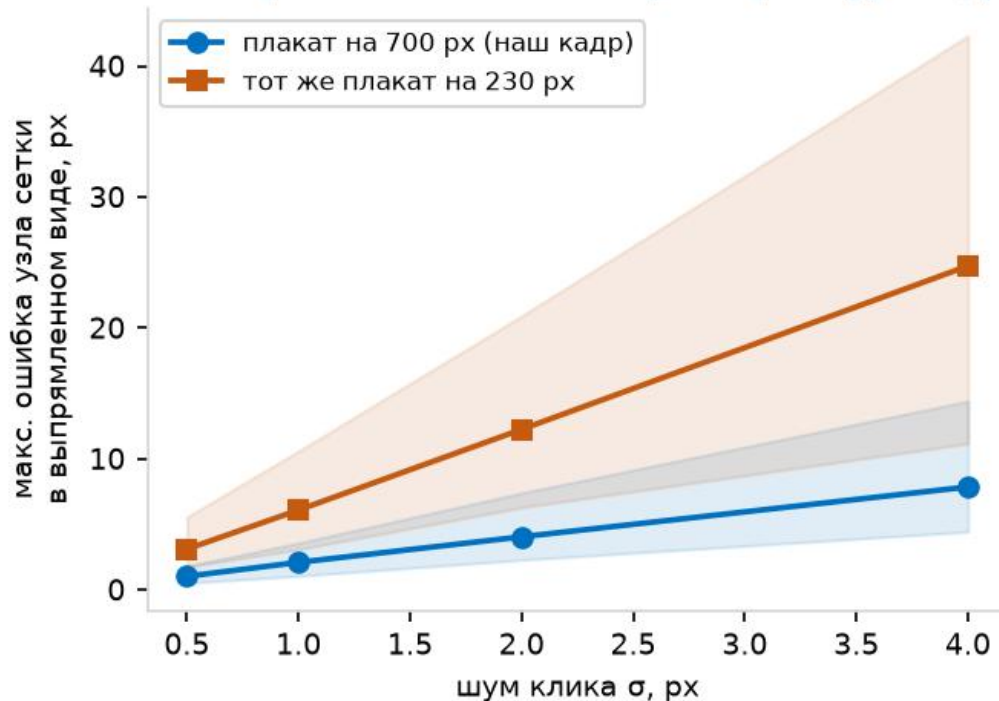
Параметр	Смысл	Типичное	Что если иначе
M / H / map_x, map_y	преобразование «источник → результат» (матрица) или готовая карта «результат → источник»	из getRotationMatrix2D, getPerspectiveTransform	WARP_INVERSE_MAP — вы передали уже обратную матрицу
dsize	размер результата (ширина, высота)	по повёрнутым углам или размер листа	shape[:2] — стороны перепутаны; мал — обрезка молча
flags	интерполяция (+ WARP_INVERSE_MAP)	INTER_LINEAR; маски — INTER_NEAREST	INTER_AREA → молча LINEAR; CUBIC ×2 времени, LANCZOS4 ×5
borderMode, borderValue	что писать за краем источника	BORDER_CONSTANT + маска; REFLECT_101 для фильтров	REPLICATE рисует полосы; константа 0 — чёрные клинья
dst= + BORDER_TRANSPARENT	рисовать поверх существующего холста	склейка нескольких кадров	без него холст затирается

Критические ситуации, когда математика «ломается» из-за неудачно выбранных точек, и вместо нормального изображения алгоритм выдает мусор или «взрыв» из пикселей:

- **Три точки аффинного на одной прямой** — система вырождена: обусловленность 112 при отклонении 10 px, **1370** при 0.1 px; `getAffineTransform` вернёт мусор без исключения
- **Четыре точки гомографии, любые три коллинеарны** — то же; точки близко друг к другу — короткий рычаг: втрое меньший плакат → ошибка $\times 3$
- **Горизонт внутри результата:** $w \rightarrow 0$, координаты $\rightarrow \infty$; bird's-eye «до горизонта» не существует — строка 186 кадра есть бесконечность
- **Как заметить по числам:** $|\det|$ близок к нулю, `pr.linalg.cond` растёт, углы результата «улетают» за тысячи пикселей

Вырождение не бросает исключение — оно возвращает картинку, которая выглядит как взрыв. Проверьте определитель и углы

ошибка растёт линейно с σ и обратно размеру в кадре



клики ± 4 px: узлы «уезжают» сильнее всего у дальнего края



Правило «четырёх точек достаточно» проверено: шум клика σ распространяется на узлы сетки выпрямленного вида линейно — 4 px при $\sigma = 2$ для плаката шириной 700 px в кадре, 12 px для того же плаката шириной 230 px. Справа: при $\sigma = 4$ px узлы уезжают сильнее всего у дальнего края

Точность выпрямления перспективы (гомографии) зависит от человеческого фактора (шума при кликах мышкой по углам объекта) и от размера самого объекта в кадре

© фото — здание Университета Иннополис; собственная иллюстрация

- **PSNR к эталону** — когда ответ известен (синтетика, «уменьшили → увеличили»): > 35 дБ — разницы не видно, < 25 — артефакты
- **Чего PSNR не видит:** геометрию — сдвиг всего на 1 px даёт **26 дБ**, на 3 px — 21; и освещение он считает ошибкой геометрии
- **Геометрическая метрика:** ошибка репроекции точек в пикселях — среднее и максимум; для документа — доля совпавших пикселей после бинаризации (**99.1 %**); для масок — IoU из L3
- Что хорошо: репроекция < 1 px — отлично, 1–3 px — приемлемо для скана, > 5 px — ищите ошибку в точках

- **Дисторсия объектива:** прямые в кадре кривые — четыре угла лежат на дугах, гомография «не ложится»; у веб-камеры угол уезжает на десятки px → сначала `undistort`
- **Не плоскость:** человек перед плакатом, книга с изгибом, две стены — одной H нет; результат «плывёт» на границе плоскостей
- **Двойной ресемплинг:** выпрямили, потом повернули, потом уменьшили — три интерполяции подряд размывают; собрать $H = H_3 \cdot H_2 \cdot H_1$ и сделать **один** варп
- **Rolling shutter** из L2: вертикали наклонены — прямые стали ломаными ещё в сенсоре; **JPEG** и шум мешают кликнуть угол точнее ± 2 px

- **Камера смотрит в лоб**, стороны плаката параллельны с точностью до градуса — хватит `срезки` и ``resize`` из L1: подобие, две пары точек, никаких «бабочек»
- **Поворот на $90^\circ/180^\circ$** — `cv2.rotate` и `cv2.flip`: перестановка пикселей без интерполяции, без потерь, за доли миллисекунды
- **Признак, что пора брать гомографию**: противоположные стороны объекта в кадре **сходятся** — угол между ними больше $1-2^\circ$, или длины отличаются больше чем на 5 %
- **Признак, что и её мало**: прямые в кадре кривые (дисторсия) или объект не плоский

Робототехника

- bird's-eye view для планировщика и парковки: метры из пикселей за 4 мс на кадр; доверять ближней зоне
- гетар с готовой картой: дисторсия и ректификация стерео на каждом кадре — L11, L12
- маркеры, QR, таблички под углом: выпрямить — потом читать

МОИИ

- сканер документов и OCR: гомография + бинаризация L3 — весь классический пайплайн
- аугментации: аффинные и проективные варпы, эластичные гетар; маски — NEAREST, иначе метки «серые»
- spatial transformer и dense flow — та же карта map_x, map_y, только выученная

- Варп выполняется **обратно**: для каждого пикселя результата — координаты источника через H^{-1} , деление на w , интерполяция, граница; гетар делает то же по любой карте
- Флаг выбирается по данным: NEAREST для масок, LINEAR по умолчанию, CUBIC/LANCZOS4 — увеличение за 2–6× времени; при уменьшении — размыть или `resize(AREA)`, в варпе AREA нет
- Открытый вопрос: сегодня H задана четырьмя кликами с ошибкой ± 2 px; как найти её по сотне автоматических соответствий с выбросами — **L10**; почему размытие спасает от алиасинга — **L6**

Демо 2.1 — прямой варп: дыры

```
1 src = cv2.resize(img, (960, 640), interpolation=cv2.INTER_AREA)
2 h, w = src.shape[:2]
3 M = cv2.getRotationMatrix2D((w / 2, h / 2), 20, 1.6)
4 c = cv2.transform(np.float32([[0, 0], [w, 0], [w, h], [0, h]]), M)[0]
5 M[:, 2] -= np.floor(c.min(0)) # холст по повернутым углам
6 size = tuple(np.int32(np.ceil(c.max(0)) - np.floor(c.min(0))))
7 p = np.c_[np.mgrid[:h, :w][::-1].reshape(2, -1).T, np.ones(h * w)] @ M.T
8 xd, yd = np.round(p).astype(int).T # forward: куда летит пиксель
9 hit = np.zeros((size[1] + 1, size[0] + 1), bool); hit[yd, xd] = True
10 region = cv2.warpAffine(np.ones((h, w), np.uint8), M, size) > 0
11 holes = 100 * (~hit[:size[1], :size[0]] & region).sum() / region.sum()
12 print("прямой варп: %.0f %% дыр; обратный (warpAffine) — 0 %%" % holes)
```

python

прямой варп: 61 % дыр; обратный
(warpAffine) — 0 %

Вызов

`cv2.transform(corners, M)`

`np.c_[mgrid..., 1] @ M.T → round`

`~hit & region`

Что делает

аффинное к четырём углам — по ним размер холста и сдвиг, чтобы всё влезло

прямой варп «в лоб»: каждый пиксель источника летит в округлённую клетку

клетки, где объект должен быть, но никто не прилетел, — дыры

Демо 2.2 — свой билинейный remap

python

```
1 def bilinear(img, mx, my):                # ★ звёздочка занятия 4
2     x0, y0 = np.floor(mx).astype(int), np.floor(my).astype(int)
3     fx, fy = (mx - x0)[..., None], (my - y0)[..., None]
4     top = img[y0, x0] * (1 - fx) + img[y0, x0 + 1] * fx
5     bot = img[y0 + 1, x0] * (1 - fx) + img[y0 + 1, x0 + 1] * fx
6     return np.uint8(np.round(top * (1 - fy) + bot * fy))
7 A = cv2.invertAffineTransform(cv2.getRotationMatrix2D((480, 320), 10, 1.3))
8 ys, xs = np.mgrid[:640, :960].astype(np.float32)
9 mx, my = [np.float32(a * xs + b * ys + c) for a, b, c in A] # dst → src
10 ref = cv2.remap(src, mx, my, cv2.INTER_LINEAR)
11 d = np.abs(bilinear(src, mx, my).astype(int) - ref)
12 print("против cv2.remap: макс %d, среднее %.2f кода" % (d.max(), d.mean()))
```

против cv2.remap: макс 4, среднее 0.12 кода
время (печатает тот же скрипт): ≈ 130 мс
против 1.0 мс у cv2.remap

Вызов

`np.floor(mx), mx - x0`

`cv2.invertAffineTransform(M)`

`cv2.remap(src, mx, my, INTER_LINEAR)`

Что делает

целая часть — левый верхний сосед, дробная — веса; четыре соседа через индексацию массивом

обратная матрица 2×3 : карта `mx`, `my` считается «результат → источник», как внутри `warpAffine`

эталон; расхождение ≤ 4 кодов — OpenCV считает веса в фиксированной точке (5 бит)

Демо 2.3 — четыре интерполяции в числах

```
1 from cvcourse.metrics import psnr
2 page = cio.imread("demo/text-page.png", "gray")
3 small = cv2.resize(page, (160, 120), interpolation=cv2.INTER_AREA)
4 crop = src[230:350, 330:490]
5 tiny = cv2.resize(crop, (20, 15), interpolation=cv2.INTER_AREA)
6 for name in ("INTER_NEAREST", "INTER_LINEAR", "INTER_CUBIC", "INTER_LANCZOS4"):
7     f = getattr(cv2, name)
8     up_t = cv2.resize(small, (640, 480), interpolation=f)
9     up_p = cv2.resize(tiny, (160, 120), interpolation=f)
10    print("%-15s текст x4: %.1f дБ · фото x8: %.1f дБ · %.2f мс"
11          % (name, psnr(page, up_t), psnr(crop, up_p),
12            ms(lambda: cv2.resize(small, (640, 480), interpolation=f))))
```

python

INTER_NEAREST — текст x4: 17.2 дБ · фото x8:
19.9 дБ · 0.04 мс
INTER_LINEAR — 17.3 · 20.6 · 0.05 мс ·
INTER_CUBIC — 17.9 · 21.8 · 0.14 мс ·
INTER_LANCZOS4 — 17.9 · 22.1 · 0.38 мс

Вызов

`cv2.resize(..., INTER_AREA)` вниз, затем флаг вверх

`psnr(a, b)` из `cvcourse.metrics`

`getattr(cv2, name)`

Что делает

честный эксперимент: эталон известен, измеряем только увеличение

PSNR: > 35 дБ — не отличить, < 25 — артефакты видны

флаг по имени — чтобы печатать то же имя, что в документации

Демо 2.4 — «AREA» в варпе не работает

```
python
1 from PIL import Image
2 size = (480, 320) # x¼ от 1920 × 1279
3 pil = Image.fromarray(img[... , :-1]).resize(size, Image.LANCZOS)
4 ref = np.asarray(pil)[... , :-1] # чужой эталон, как в L1
5 M = np.float32([[0.25, 0, -0.375], [0, 320 / h, (320 / h - 1) / 2]])
6 lin = cv2.warpAffine(img, M, size, flags=cv2.INTER_LINEAR)
7 area = cv2.warpAffine(img, M, size, flags=cv2.INTER_AREA) # молча LINEAR
8 blur = cv2.warpAffine(cv2.GaussianBlur(img, (0, 0), 1.5), M, size)
9 d = np.abs(lin.astype(int) - area).max()
10 print("warpAffine LINEAR %.1f дБ · «AREA» %.1f дБ · Δ между ними %d кодов"
11       % (psnr(ref, lin), psnr(ref, area), d))
12 print("GaussianBlur σ 1.5 → warpAffine LINEAR: %.1f дБ" % psnr(ref, blur))
```

warpAffine LINEAR 27.9 дБ · «AREA» 27.9 дБ ·
Δ между ними 0 кодов
GaussianBlur σ 1.5 → warpAffine LINEAR: 36.7
дБ · (cv2.resize INTER_AREA — 37.2 дБ, демо
2.2 лекции 1)

Вызов

`Image.resize(size, LANCZOS)` из Pillow

`M = [[0.25, 0, -0.375], ...]`

`cv2.GaussianBlur(img, (0, 0), 1.5)`

Что делает

независимый эталон уменьшения — как в демо 2.2 лекции 1

масштаб ¼ с поправкой на полпикселя: центр пикселя результата — в центре блока 4 × 4 источника

антиалиасинг до варпа: ядро по σ; 1.5 подобрано измерением для ×4

Фрагмент «выпрямить и перенести маску»

- `src = np.float32([tl, tr, bl, br])`
- `dst = np.float32([[0, 0], [W, 0], [W, Hh], [0, Hh]])`
- `H = cv2.getPerspectiveTransform(src, dst)`
- `out = cv2.warpPerspective(img, H, img.shape[:2])`
- `m_out = cv2.warpPerspective(mask, H, (W, Hh))`

Три ошибки

- порядок точек: в `src` — `tl, tr, bl, br`, в `dst` — `tl, tr, br, bl`: две пары перепутаны, лист сложится «бабочкой»
- `dsize = img.shape[:2]` — это `(H, W)` исходного кадра, а нужен размер листа `(W, Hh)`: результат обрезан и перевёрнут по осям
- маска варпится с флагом по умолчанию `INTER_LINEAR`: появятся 136 «серых» уровней; нужен `INTER_NEAREST`

ПОДВОДНЫЕ КАМНИ

- **`dsize` — (ширина, высота)**, а **shape** — (высота, ширина); маленький **dsize** обрезает молча — поворот на 30° теряет 18 % пикселей
- **Порядок точек** в **src** и **dst** должен совпадать попарно — **tl, tr, br, bl**; перепутали — «бабочка» без единого предупреждения
- **Знак угла**: **getRotationMatrix2D** — положительный угол против часовой стрелки, хотя ось **y** смотрит вниз; центр — в **(x, y)**, не **(row, col)**
- **Маска через `INTER\_LINEAR`** получает 136 уровней вместо двух; метки классов, глубину и маски ресемплить только **INTER_NEAREST**

ПОДВОДНЫЕ КАМНИ

- ``INTER_AREA`` в ``warpAffine``/``warpPerspective``/``remap`` молча становится ``LINEAR``: при уменьшении $\times 4$ — 27.9 дБ вместо 37; размыть заранее или `resize`
- **Гомография — только для плоскости и без дисторсии**: сначала `undistort` (угол уезжает на десятки px), человек перед плакатом «поплывёт», две стены — две H
- **Карты ``remap`` только ``float32``**: `np.mgrid` и арифметика с `float64` матрицей дают `float64` → исключение; `convertMaps` для скорости
- **Несколько варпов подряд** размывают картинку столько же раз: собрать одну матрицу произведением и варпить один раз; клики ± 2 px $\rightarrow$ 4 px ошибки — точки брать далеко друг от друга

KEY TAKEAWAYS

- Однородные координаты делают сдвиг, поворот, аффинное и перспективу **одной матрицей 3×3** ; иерархия $2 \rightarrow 3 \rightarrow 4 \rightarrow 6 \rightarrow 8$ DoF, пар точек нужно DoF/2: аффинному — 3, гомографии — 4
- Каждая ступень что-то теряет: евклидово хранит длины, подобие — углы, аффинное — параллельность (площади $\times \det A = 0.93$), гомография — только прямые и двойное отношение ($1.25 = 1.25$, а AB/BC $0.5 \rightarrow 0.416$)
- Варп выполняется **обратно**: прямой оставляет 61 % дыр; OpenCV для каждого пикселя результата берёт H^{-1} , делит на w и интерполирует — 1920×1279 за 3–4 мс с LINEAR, 19 мс с LANCZOS4
- Флаг — по данным: NEAREST для масок (иначе 136 «серых» уровней), LINEAR по умолчанию, CUBIC/LANCZOS4 +2 дБ на увеличении фото; при уменьшении в варпе **нет** AREA — размыть $\sigma \approx 1.5$ ($27.9 \rightarrow 36.7$ дБ) или `resize`
- Гомография законна для плоскости без дисторсии; клик ± 2 px $\rightarrow$ 4 px ошибки, втрое меньший объект — 12 px; bird's-eye растягивает 20 м в 6 раз; в L10 H будем не задавать, а оценивать по соответствиям

САМОПРОВЕРКА

- Сколько пар точек нужно аффинному преобразованию и гомографии — и что именно добавляют два лишних параметра h_{31} , h_{32} ?
- Кадр 1920×1279 поворачивают на 30° вокруг центра: какой `dsize` сохранит все пиксели, как сдвинуть матрицу и сколько процентов холста останется пустым?
- Маску сегментации повернули `warpAffine` с флагом по умолчанию: что в ней появилось, как проверить одной строкой NumPy и как исправить?
- Bird's-eye view на 20 м: почему штрихи разметки «мылятся», хотя ширина полосы измеряется точно — и что изменит камера с вдвое большим f_x ?

Разбор — в начале L5, на слайде «Опоры»

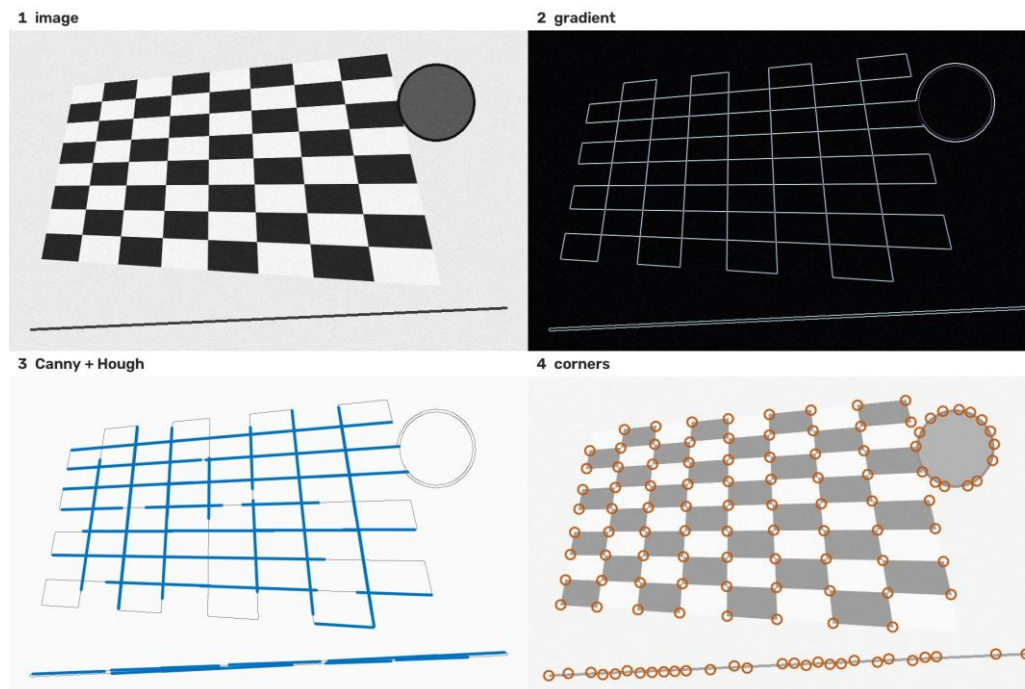
REFERENCES

- Szeliski R. *Computer Vision: Algorithms and Applications*, 2nd ed., 2022 — §2.1.1–2.1.2 (2D-преобразования и иерархия), §3.6 (геометрические преобразования, forward/inverse warping, интерполяция). К следующей паре: §3.2 (линейная фильтрация)
- Hartley R., Zisserman A. *Multiple View Geometry in Computer Vision*, 2nd ed., 2004 — гл. 2: проективная геометрия плоскости, таблица инвариантов 2.1, точки на бесконечности
- Heckbert P. *Fundamentals of Texture Mapping and Image Warping*, M.Sc. thesis, UC Berkeley, 1989 — откуда пошёл обратный варп и ресемплинг
- Keys R. *Cubic Convolution Interpolation for Digital Image Processing*, IEEE TASSP, 1981 · Duchon C. *Lanczos Filtering*, J. Appl. Meteorology, 1979 — ядра слайда 32
- OpenCV docs 4.x: *Geometric Image Transformations* — warpAffine, warpPerspective, remap, getRotationMatrix2D, getPerspectiveTransform, convertMaps, initUndistortRectifyMap
- Гонсалес Р., Вудс Р. *Цифровая обработка изображений*, 3-е изд., 2012 — §2.4.4 (интерполяция), §2.6.5 (геометрические преобразования)

ТОЛКОВЫЙ СЛОВАРЬ

Термин	Что это значит
однородные координаты (homogeneous coordinates)	точка (x, y) как тройка (x, y, w) с точностью до множителя; сдвиг и перспектива становятся матрицами
точка на бесконечности (point at infinity)	тройка с $w = 0$ — направление; туда уходят параллельные прямые, оттуда берётся точка схода
степени свободы (degrees of freedom, DoF)	число независимых параметров преобразования; пар точек для его поиска нужно $\text{DoF}/2$
евклидово преобразование (rigid transform)	поворот и сдвиг, 3 DoF; сохраняет длины и углы
подобие (similarity)	евклидово плюс масштаб, 4 DoF; сохраняет углы и отношения длин
аффинное преобразование (affine transform)	любая матрица 2×2 плюс сдвиг, 6 DoF; сохраняет параллельность, площади $\times \det$
гомография (homography)	проективное преобразование плоскости, 3×3 до масштаба, 8 DoF; сохраняет только прямые
точка схода (vanishing point)	точка кадра, в которой сходятся образы параллельных прямых плоскости
двойное отношение (cross-ratio)	$(AC \cdot BD)/(BC \cdot AD)$ четырёх точек прямой — инвариант гомографии; простые отношения не сохраняются
прямой варп (forward warping)	пиксели источника летят в результат; оставляет дыры и столкновения
обратный варп (inverse warping)	для каждого пикселя результата ищется точка источника и интерполируется; так работает OpenCV
интерполяция (interpolation)	оценка значения между узлами по соседям; ядро задаёт веса: 1, 4, 16 или 64 соседа
алиасинг (aliasing)	ложный узор при уменьшении без усреднения: часть отсчётов выброшена; лечится размытием до варпа
bird's-eye view	вид сверху на плоскость дороги по гомографии; метры точны вблизи, дальше растянута

Вопросы?



Dr. Ilya Afanasyev · i.afanasyev@innopolis.ru

Демо-код лекции: [lectures/L04-geometry/demo/](#)

Занятие 4 — принесите фото плаката, этикетки или листа под углом